

Aimath Reference Book

The model proposes. Lean decides. Mathlib is the trusted library. Everything else is a hint. Local novelty is not a claim of a new mathematical truth.

Read from Chapter 1 forward. This file is the concatenated source of the numbered book only.

Chapter 1 — How to read this book

This book is the long explanation of aimath: a program you run on your own computer that tries to do mathematics in a way a machine can check. The book is written for a reader who may never have heard of Lean, Mathlib, theorem provers, or language-model agents. It is also written for a reader who already lives in those worlds and wants every knob, file, status word, and invariant named without apology. Those two readers are not given two books. They are given one book that starts from the first idea and does not skip the minutes.

If you are completely new, read from this page forward and do not skip the chapters that look “too basic.” The later chapters use the vocabulary of the early ones on purpose. If you already know Lean, still read Part I until the trust rule is boring. Most mistakes with this system are not technical. They are mistakes about who is allowed to call something a theorem.

What this book is

It is a reference book, a manual, and a course of explanation in one spine.

- A reference because later chapters list commands, configuration keys, files, protocol messages, and source modules in enough detail that you should not need to guess.
- A manual because it tells you what to type, what should appear, and what each outcome means.
- A course because it does not assume you already know what a formal proof is, what a type is, what Mathlib is, or why sorry is not a proof.

It is meant to be the ultimate written source for this repository. The short README at the root of the project is a front door. The audience pamphlets under docs/audiences/ are invitations in different tones. This book is the place the pamphlets are allowed to point when a reader says “explain everything.”

What this book is not

It is not a textbook of mathematics. It will use small facts about natural numbers because those facts are easy to state and easy for Lean to check. It will not teach you calculus, algebra, or analysis. If you do not already know what “for all natural numbers n ” means, you can still install the software and run the examples, but you will be watching a machine check sentences you cannot yet interpret. That is allowed. It is not the same as understanding.

It is not a history of Leibniz, Lean, or artificial intelligence. Historical names appear because they explain design choices, not because this book is a biography.

It is not a promise that the software will discover new mathematics. The software can store a Lean-checked proof that is new to this installation’s notebook. That is a precise, small claim. The book will repeat that distinction until it is impossible to miss.

It is not a substitute for the Lean kernel. If this book and Lean disagree about whether a file is a proof, Lean is right and the book is wrong.

The one rule the whole book exists to protect

A language model never decides that something is a theorem.

Lean decides.

Mathlib is the trusted library.

Everything else — Wikipedia, OEIS, arXiv abstracts, your notes, Isabelle or Coq excerpts, the model's English, a personality named after Euler — is a hint.

If you remember only one paragraph, remember that one.

How the parts fit together

The book has a plot, even though it is technical.

Part I answers: what would it even mean for a machine to do mathematics carefully? It starts with ordinary school proofs, moves to formal proofs, then to Lean, Mathlib, and language models. You should finish Part I able to say, in your own words, why a fluent paragraph from a chatbot is not a certificate.

Part II answers: what did we actually build? You meet the trust model, the files on disk, the install, and the configuration file. You should finish Part II able to point at `aimath.yaml`, `lean_ws/`, and `aimath.sqlite` and say what each is for.

Part III answers: what do I type? You work through the first hour, then every command. You should finish Part III able to choose among `prove`, `search`, `explore`, `swarm`, `curious`, `research`, `system`, `report`, `serve`, and `worker` without treating them as synonyms.

Part IV answers: what may I say after the machine runs? Novelty language, open problems, personalities. You should finish Part IV unwilling to tell a friend “we proved the Riemann hypothesis” because a model emitted tactics.

Part V answers: how does it use the machine, other machines, and the source tree? Budgets, sockets, SQLite, modules.

Part VI answers: how should different people use this, and how should it be changed without breaking the rule?

The appendices are for lookup after you have read the spine once.

Typographical conventions

Commands you type appear like this:

```
aimath prove --statement "forall n : Nat, n + 0 = n"
```

On Windows this is PowerShell unless the text says otherwise. On other systems the same command usually works in bash or zsh. Where Windows is special (elan install, curl versus curl.exe), the book says so.

File paths appear like lean_ws/ and aimath.yaml. They are relative to the directory where you ran aimath init, unless the text says they live inside the source repository.

Status words are written in roman type when they are outcomes: theorem, known, conjecture, rejected. Those four words are not informal English. They are the machine’s vocabulary. Chapter 18 treats them as a contract.

When the book quotes Lean, it uses Lean’s own keywords: theorem, lemma, def, axiom, sorry, admit, by, import.

When the book says relative to system S, it means the proof used extra axioms that are not part of ordinary Mathlib mathematics. That phrase is not decoration. It is a warning label.

How to work while you read

You can read the whole book before installing anything. Part I is designed that way. From Part II onward, a machine helps. The intended loop is:

1. Read a section.
2. Type the command it names, if you have the tools.
3. Compare what you see with what the book said you would see.
4. If they differ, believe the program’s output about your machine, then read the troubleshooting appendix.

If you cannot install Lean yet, still read. The trust rule does not require a download.

Voice and pedantry

This book is deliberately slow. It will define “terminal,” “PATH,” “environment variable,” “process,” and “JSON line” when those words first become necessary. It will also define “proposition,” “quantifier,” and “axiom.” The author would rather bore an expert for a page than leave a beginner stranded for a chapter.

Experts are asked to treat the slow pages as a specification of what the project believes a new user must be told. If a later contributor shortens the install chapter until a beginner cannot finish it, that is a documentation regression, not a cleanup.

Version

The software described is aimath 0.1.x as it exists in this repository: a Python package, a Lake project template that requires Mathlib, SQLite storage, optional web hints, and a localhost-first worker protocol. Features that were explicitly left out of the first plan (full literature search, Aesop-scale proof search, a daemon that researches while you sleep, a paper-quality PDF of original theorems) are named as absences in Chapter 29. The book does not pretend they are present.

A note on names

aimath is the command and the Python package. Lean 4 is the proof assistant. Mathlib is the community library. elan is the version manager that puts lean and lake on your PATH. Lake is Lean's build tool. An LLM is a language model: a program that predicts text. In this project it predicts Lean text and English justifications. It is not a kernel.

Named personalities (euler, noether, erdos, grothendieck, ramanujan) are prompt styles. They are not simulations of those people and they confer no historical authority on a file.

If you are in a hurry anyway

The honest short path is:

1. Chapter 2 (what a proof is) and Chapter 8 (the trust model).
2. Chapter 10 (install).
3. Chapter 12 (first hour).
4. Chapter 18 (status words).
5. Chapter 19 (what you may claim).

Then come back and read the rest. The short path is not the book. It is a concession.

Dedication of attention

This book is dedicated to the difference between a sentence that sounds like mathematics and a sentence that has been checked. The software exists to keep that difference visible. The book exists so that a human being, looking at the software, does not blur it again.

Chapter 2 — What a proof is, beginning from school

Before this book talks about Lean, it talks about ordinary proofs. If you skip this chapter, later chapters will sound like a religion: “the kernel said so.” This chapter is why anyone should care what a kernel says.

A claim

Here is a claim you may have seen in school:

> For every natural number n , $n + 0 = n$.

The natural numbers, in the school sense, are 0, 1, 2, 3, and so on. The claim says: if you take any one of those numbers and add zero, you get the same number back.

Is the claim true? Most people say yes. Why? Because “adding zero does nothing.” That is an explanation. It is not yet a proof in the sense this book needs, but it is the right kind of start: it tries to say why, not merely to repeat the claim more loudly.

An argument that would pass in a classroom

A teacher might accept this:

> Zero is the additive identity of the natural numbers. By the definition of addition, $n + 0$ is n . Therefore the claim holds for every n .

A stricter teacher might want induction:

> Base case: $0 + 0 = 0$.

> Inductive step: if $k + 0 = k$, then $(k+1) + 0 = (k + 0) + 1 = k + 1$.

> By induction the claim holds for all natural numbers.

Both arguments have the shape of a proof: they start from things the class has already agreed to (definitions, earlier lemmas, the induction principle) and they arrive at the claim by steps the class has already agreed are legal.

That last sentence is the whole subject of formal mathematics. A proof is not a feeling. It is a chain of allowed steps from allowed starting points.

An argument that would not pass

> I asked a clever friend and the friend said it was true.

> I tried $n = 0, 1, 2, 3, 4$ and it worked, so it is always true.

> The equation looks balanced.

> A computer printed “True.”

The second of those is interesting. Checking examples is good science and good habit. It is not a proof of a “for all n ” claim unless you have a theorem that says those examples are enough. Five examples are not every natural number.

The last of those is the trap this software exists to avoid. Computers print “True” for many reasons: a bug, a heuristic, a language model imitating a textbook, a test that only checked the first case. This book will not treat a printed word as a proof.

What “allowed steps” means

In a classroom, allowed steps are social. The teacher and the textbook say what you may use. Different classrooms disagree. One class has already proved that addition is commutative. Another has not, and must not use that fact yet.

In a computer, allowed steps can be made mechanical. You fix:

1. A language in which claims and steps are written.
2. A list of starting sentences (axioms and definitions).
3. A list of rules that turn old sentences into new ones.
4. A program that checks that each step is one of those rules.

If the program accepts a text, then — relative to those axioms, definitions, and rules — the last sentence follows. That is a formal proof.

Formal is not a compliment. It means “according to a form.” A formal proof can be about a silly axiom system. It can prove things that are false in the physical world if you chose silly axioms. Its virtue is not wisdom. Its virtue is that the check does not depend on the author’s confidence.

Why formal proofs matter if you already “know” the math

They matter when:

- The argument is long and humans lose track of cases.
- The definitions are delicate (what is a real number, a manifold, a scheme).
- Two people disagree and shouting will not settle it.
- A machine proposed the argument, and machines are fluent liars.

aimath is built for the last case especially. A language model can write a paragraph that looks like the inductive proof above and still be wrong in a way a tired human misses. A kernel does not get tired in that way. It also does not understand. It only checks form.

Propositions

A proposition is a sentence that can be true or false.

“7 is prime” is a proposition.

“Hello” is not.

“ $n + 0 = n$ ” is a proposition once n is given a type, or it is a proposition with a hidden “for all n ” depending on how you write it. Precision about that is not pedantry. It is the difference between a statement Lean can elaborate and a string of English the model liked.

In Lean, the type of propositions is called `Prop`. You do not need to master type theory to use aimath, but you need this much: the software will try to treat your `--`statement as a `Prop`. If it cannot, the status will be rejected, and that rejection is a kindness. It means the sentence was not even a well-formed claim yet.

Quantifiers, said slowly

For all (written `forall` or $\forall$) says: no matter which object of this kind you pick, the rest is true.

There exists (written `exists` or $\exists$) says: at least one object of this kind makes the rest true.

School English hides quantifiers. “The sum of two evens is even” means: for all integers a and b , if a is even and b is even, then $a+b$ is even. If you forget the “for all,” you have a different sentence.

aimath does not fix your quantifiers for you. If you write a weaker sentence, it may prove a weaker sentence and you may think you proved the stronger one. That is a human error the kernel cannot catch, because the weaker sentence is still a real theorem.

Equality

“ $=$ ” in Lean is a proposition: $a = b$ is something that can have a proof. It is not an assignment like $a = a + 1$ in some programming languages. Lean is both a programming language and a proof language. In the proofs this book cares about, $=$ is the mathematical relation.

`rfl` (reflexivity) proves $a = a$ when both sides are already the same after Lean’s definitional computation. Many “obvious” equations are `rfl` once definitions unfold. Many others are not. The model loves to say `rfl` when it is not enough. Lean will refuse.

Induction, informally

Induction is a legal way to prove “for all natural numbers n , $P(n)$ ” if you prove $P(0)$ and you prove that $P(k)$ implies $P(k+1)$. School courses sometimes treat induction as a magic template. In Lean it is a lemma or a tactic arising from how `Nat` is defined: a natural number is zero or the successor of a natural number.

`aimath` may or may not invent a correct induction. You should still know what induction is, so that you can tell a good script from a cargo-cult script that mentions “induction” and then uses `sorry`.

sorry as a moral object

In Lean, `sorry` fills a hole. It lets a file elaborate while a proof is unfinished. It is a wonderful tool for humans writing a development. It is a disaster if you call the hole a theorem.

`aimath` treats `sorry` and `admit` as banned identifiers in sources it is willing to accept. A comment containing the word is allowed. A string containing the word is allowed. A live tactic `sorry` is not a proof. This is not optional and not configurable in 0.1.x. If a later version adds a “draft mode,” it must not write the status theorem. That would be a betrayal of Chapter 1.

Two kinds of certainty people confuse

Mathematical certainty (informal): the community of mathematicians is convinced.

Kernel certainty (formal): a specific checker accepted a specific file in a specific axiom system.

These can come apart.

A kernel-accepted file can formalize a statement nobody finds interesting.

A community-accepted theorem can wait years to be formalized.

A kernel-accepted file in a silly axiom system can “prove” almost anything.

`aimath` only ever offers the second kind, plus a notebook of what this machine has seen. When humans say “we proved it,” they usually mean the first kind. Do not launder the second into the first.

What you should be able to say after this chapter

- A proof is a chain of allowed steps from allowed starts, not a vibe.
- Examples are not proofs of universal claims.
- Formal proofs make “allowed” mechanical.
- Formal proofs are relative to axioms.
- `sorry` is a hole.
- A chatbot's fluency is not a checker.

The next chapter takes the same ideas into the language Lean actually uses, still slowly.

Chapter 3 — Formal mathematics and a language precise enough to check

Chapter 2 said a formal proof needs a language, starting sentences, rules, and a checker. This chapter names those pieces as they appear in the world aimath lives in.

A language that is not English

English is a wonderful language for teaching and a bad language for kernels. The sentence “every space is compact if it is finite” is ambiguous until you say what a space is. The sentence “let G be a group” hides pages of definition.

A formal language fixes syntax. Lean’s syntax is closer to a programming language than to English. You write:

```
∀ n : Nat, n + 0 = n
```

or, in ASCII that aimath is happy to accept on the command line:

```
forall n : Nat, n + 0 = n
```

That string has parts:

- `forall n` — a binder: we are speaking about every n .
- `: Nat` — n is a natural number in Lean’s sense, not a real, not an integer, not a school word.
- `,` — then the body.
- `n + 0 = n` — a proposition about that n .

If you write `forall n, n + 0 = n` without a type, Lean may or may not infer `Nat`. If it infers something else, you proved something else. Always think about types.

Types, without a course in type theory

A type is a kind of object.

`Nat` is the type of natural numbers.

`Int` is the type of integers.

`n + 0` only makes sense if `+` is defined for the type of n .

`True` and `False` are propositions (in fact they are in `Prop`). They are not numbers.

Beginners lose hours because they write a true-sounding sentence at the wrong type. “Every number is even or odd” is false for real numbers and true for integers, with the usual definitions. Lean will not guess which number you meant.

aimath will import Mathlib and ask Lean to elaborate your statement as a Prop. If elaboration fails, you get rejected with Lean’s error. Read the error. It is often “unknown identifier” or a type mismatch. It is almost never “the AI is unwise.”

Definitions are not theorems

A definition names a thing.

```
def double (n : Nat) : Nat := n + n
```

This does not claim a fact about the world. It says what double means.

A theorem claims a fact:

```
theorem double_eq (n : Nat) : double n = n + n := by rfl
```

aimath’s research command asks the model for a definition and a consequence. It keeps the definition only if the consequence checks. That is a design prejudice: new notation must earn its keep by proving something. A definition that merely exists is not a result.

Axioms are not theorems either

An axiom is a starting sentence you are willing to assume.

Mathlib’s ordinary developments use a standard foundation (dependent type theory with the axioms Lean and Mathlib already commit to). You should treat “a Mathlib theorem” as a theorem in that foundation.

When you load a YAML file of extra axioms, you are adding starts. Anything you prove after that is a theorem in Mathlib plus those axioms. It may be false in ordinary mathematics. It may be consistent or inconsistent. Lean will not tell you that the axioms are wise. It will only tell you that the file checks.

Chapter 7 and Chapter 16 return to this until the word “relative” is automatic.

Tactics

A Lean proof can be written as a term (a program that inhabits the proposition) or as a tactic script (instructions that build the term).

aimath almost always wants tactics, because language models are better at emitting simp, ring, intro, rfl than at emitting full proof terms.

A tiny script:

```
theorem demo :  $\forall n : \text{Nat}, n = n := \text{by}$   
  intro n  
  rfl
```

intro n moves the universal quantifier into a local hypothesis. rfl closes an equality that is definitionally true.

The by block is the proof. Everything after := by is what aimath asks the model to write when you prove.

The kernel and the elaborator

Lean is not one blob.

The elaborator turns the text you wrote — implicit arguments, tactics, notations — into a fully explicit term.

The kernel checks that the term is well-typed and that it has the claimed type (the proposition).

If a tactic is buggy in a way that produces a bad term, the kernel should refuse. aimath's extra sorry scan is a belt in addition to the kernel: even if someone tried to smuggle an unfinished proof, the file is rejected before we celebrate.

You do not need to call the kernel yourself. lake env lean file.lean does the work. aimath runs that.

What “Mathlib” is doing in this story

If Lean is the language and the checker, Mathlib is the enormous pile of already-checked definitions and theorems written in that language: groups, topology, analysis, number theory, and far more.

Without Mathlib, you can still write Lean, but you will reinvent numbers and struggle. aimath therefore requires Mathlib. It is not an optional plugin in 0.1.x. Init downloads it. Prove will not start without it.

That requirement has a cost: disk, time, RAM. The book will not hide the cost. It is the price of not working in a toy language.

What a language model is doing in this story

A language model is a statistical machine that continues text. If you show it a lot of Lean and a lot of English, it will emit more Lean-shaped and English-shaped text.

It does not have a kernel inside it.

It will invent lemma names that do not exist.

It will emit sorry if you let it.

It will claim to have proved open problems.

It is still useful, the way a colleague who talks too fast is useful: you take notes, then you check.

aimath's entire architecture is: let the model talk; let Lean grade; write down only what Lean passed.

Why this is not “AI that does math” in the movie sense

Movies want a machine that announces new truths. This system announces:

- Lean accepted this file, or
- Lean did not, or
- we already had this in Mathlib or in the notebook, or
- this text names a famous open problem and we will not call a tactic script a solution.

That is a smaller machine. It is the one we can defend.

A first complete picture

Imagine a desk.

On the left: a talkative intern (the model).

In the middle: a printed encyclopedia that was already refereed line by line (Mathlib).

On the right: a pedantic examiner who only reads Lean (the kernel).

In the drawer: a lab notebook for this room only (SQLite).

On the wall: newspaper clippings (Wikipedia, arXiv, your notes). They are not the encyclopedia.

aimath is the room. You are the person who decides what question is worth putting on the desk.

The next chapter says why anyone historically wanted such a room, and why we used these particular tools.

Chapter 4 — Leibniz’s three pieces, and the tools we used for them

This chapter is the only historical chapter. It exists so that the words “language,” “library,” and “reasoner” in the README are not slogans.

What Leibniz wanted

Gottfried Wilhelm Leibniz (1646–1716) imagined, at different times and in different texts:

- a *characteristica universalis*: a written language in which ideas could be expressed without the fog of ordinary speech;
- a *calculus ratiocinator*: a way of calculating with those signs so that disputes could be settled by “let us calculate”;
- an organized encyclopedia of human knowledge.

Historians argue about how systematic the dream was. This book does not need the argument. It needs the three-part shape: a language, a store of what is already known, and a process that computes with both.

Leibniz did not have Lean. He did not have Mathlib. He did not have GPUs. He had a hope that disagreement could be made mechanical. Formal proof assistants are one of the few places that hope became a daily tool rather than a metaphor.

What we did not inherit

We did not inherit a language that settles philosophy, theology, or politics. Lean will not tell you whether a definition is the right definition for a research program. It will not tell you whether a theorem is interesting. It will not tell you whether a paper should be published.

We did not inherit an encyclopedia of all mathematics. Mathlib is large and incomplete. Papers, books, and other provers contain work that is not in Mathlib. aimath can peek at some of that work as untrusted text. Peeking is not inclusion.

We did not inherit a reasoner that is a calculus in Leibniz’s sense. A language model is not a decision procedure. It is a proposal engine. The calculus, such as it is, is Lean’s kernel plus tactics like ring and omega.

If you come to this project hoping for the full Leibniz machine, you will be disappointed unless you keep the mapping honest:

Leibniz’s piece	What we actually have
---	---
Universal language	Lean 4, which is a specific type theory, not a language of all thought

| Encyclopedia | Mathlib first; a local notebook; optional clippings |
| Calculator of reasons | Tactics and a kernel; an LLM only to guess the next line |

That mapping is already ambitious. It is not the dream completed.

Why Lean 4, not a private language

We could have invented a toy syntax. That would make demos easy and mathematics impossible.

Lean 4 is a living language with:

- a small trusted kernel;
- an industrial tactic framework;
- Mathlib;
- a package manager (Lake) and a toolchain pin (lean-toolchain);
- a community that already argues about the right way to state lemmas.

aimath is parasitic on that community in the good sense. It does not fork Lean. It writes files and runs lake env lean.

Why Mathlib is required, not optional

An earlier plan sketched Mathlib as optional. That was wrong for this project's stated aim. A "library of mathematics" that is an empty folder plus Wikipedia is a scrapbook.

Mathlib is the primary library. Init creates a Lake project that requires leanprover-community / mathlib. Search looks there first. User axioms import Mathlib. The local database does not copy Mathlib into SQLite. Copying would waste space and go stale. Mathlib lives on disk under lean_ws/.lake/packages/mathlib.

Why a language model at all

Because guessing the next tactic is a place where fluent pattern-matching helps, and because stating a conjecture in Lean syntax is also a place where fluent pattern-matching helps.

The project's original request asked for thousands or millions of agents, curiosity, and personalities. Those are proposal-side ideas. They are implemented as queues of tasks and prompt styles, not as millions of operating-system processes. Chapter 15 and Chapter 20 explain that without romance.

If you have no model configured, you can still init, inspect resources, load a system, and write a report. You cannot usefully prove in 0.1.x, because the prove loop asks the model for tactics after typecheck. search may close some goals with the tactic ladder alone, but the CLI still builds a mathematician that expects a client. Configure a model, even a small local one.

Why Python around Lean

Lean is the examiner. Python is the laboratory technician: it probes RAM, talks HTTP to model servers, writes SQLite, parses YAML, opens sockets, and offers a CLI. This is not because Python is a foundation for mathematics. It is because the glue around a prover is ordinary systems work.

The trust boundary is the Lean file. Python can be buggy. If Python stores a theorem without Lean accepting the file, that is a bug you should treat as a crisis. The tests exist to make that crisis less likely. They do not make it impossible.

Why Windows is mentioned so often

This repository was developed on Windows as well as designed to be portable. Windows PowerShell aliases `curl` to `Invoke-WebRequest`. `irm` against some GitHub raw URLs fails with a closed connection. `elan`'s current documented installer lives at <https://elan.lean-lang.org/elan-init.ps1>. Those are not philosophical facts. They are the minutes that stop a beginner. Chapter 10 writes them down in order.

The original request, restated without hype

Someone asked for a system that would find new mathematical truths, using Lean, a library of all mathematics, an LLM reasoner, a huge agent prover, host-aware use of the machine, concurrent and distributed work, new formal systems, user systems, curiosity, and personalities from many fields.

What you can hold in your hands is the subset that can be defended:

- Lean-checked theorems new to this corpus, not certified new to the world;
- Mathlib as the library, plus optional untrusted extras;
- a host-capped swarm of logical agents;
- a real but small distributed check protocol;
- personalities as prompts;
- curiosity as a rotating command, not a soul;
- formal systems that stay labeled.

Chapter 29 lists what is still missing. The rest of the book teaches what is present so thoroughly that you will not confuse the two.

Transition

You now have the idea of a proof, the idea of a formal language, and the historical mapping. The next chapter is Lean from zero as a user of `aimath` needs it: enough to read a file, not enough to write a compiler.

Chapter 5 — Lean from zero, as aimath uses it

This chapter is not a Lean textbook. It is the minimum Lean a person must know to read aimath's output and to write a `--`statement that means what they think it means.

What Lean is

Lean 4 is an interactive theorem prover and a functional programming language. In the same file you can write programs that run and theorems that are checked. aimath cares almost entirely about the second use.

When this book says “Lean accepted the file,” it means: the Lean executable, running in the Mathlib workspace through Lake, processed the file and exited with code 0, and the file did not contain a live sorry or admit.

The toolchain, said slowly

You do not download “a Lean” once and forget it. You install elan, which is a version manager. elan reads a file named lean-toolchain in a project directory. That file contains one line such as:

```
leanprover/lean4:v4.35.0-rc2
```

When you run lake or lean inside that directory, elan makes sure that exact version is present. Mathlib is extremely sensitive to this pin. If your toolchain and Mathlib disagree, you get mysterious errors. aimath init rewrites lean-toolchain from Mathlib's current pin on purpose.

Lake is the build tool. lake update fetches packages listed in lakefile.toml. lake exe cache get downloads prebuilt artifacts so you do not compile all of Mathlib from scratch. lake env lean myfile.lean runs Lean with the package environment (so import Mathlib works).

aimath never asks you to click a GUI. It shells out to lake.

A file Lean can check

A minimal file in the aimath workspace looks like this:

```
import Mathlib

theorem aimath_result : (∀ n : Nat, n = n) := by
  intro n
  rfl
```

Line 1 makes Mathlib's declarations visible.

The theorem has a name (aimath_result), a statement after the colon, and a proof after := by.

If you already wrote a full theorem ... := by ... as your statement, aimath will use your text as the declaration instead of wrapping it. That is an advanced convenience. Beginners should pass only the proposition.

How aimath wraps your statement

If you type:

```
aimath prove --statement "forall n : Nat, n = n"
```

and the model returns the tactics intro n and rfl, the sandbox writes approximately:

```
import Mathlib
-- or a narrower import if retrieval found one

theorem aimath_result : (forall n : Nat, n = n) := by
  intro n
  rfl
```

The extra parentheses around the statement are deliberate. They keep precedence from surprising you.

If retrieval found Mathlib.Data.Nat.Basic, the import line may be that module instead of all of Mathlib. Narrower imports start faster. If typecheck fails with a narrow import, the prove loop falls back to import Mathlib and tries again.

Typecheck without proving

Before wasting model calls on a sentence that is not a proposition, aimath writes a dummy:

```
import Mathlib

theorem aimath_typecheck_dummy : (YOUR_STATEMENT) ∨ True := Or.inr trivial
```

Why this trick?

- YOUR_STATEMENT must be a Prop or Lean will not accept the ∨.
- Or.inr trivial proves the right disjunct, True, without proving your statement.
- The file contains no sorry.

If this file fails, your statement did not elaborate. There is nothing to prove yet. Status: rejected.

If this file succeeds, your statement is a well-typed proposition. It is still unproved. The loop then asks for tactics.

This dummy is not a proof of your claim. If you ever see it in a report as if it were, that is a misunderstanding. The report should only export accepted proofs of the real goal.

Tactics you will see constantly

rfl — reflexivity of equality after definitional unfolding.

intro / intros — introduce hypotheses and bound variables.

simp — rewrite with a database of “simp lemmas.” Powerful and sometimes too powerful.

ring — equalities in rings (including many numeral and polynomial identities).

omega — linear arithmetic on integers and naturals, in the modern Lean sense.

norm_num — close numeric goals.

linarith — linear inequalities.

aesop — a general automation tactic; may or may not be available depending on imports.

decide — try to close a decidable proposition by computation.

skip — do nothing. aimath inserts skip if the model returns an empty tactic block, so the file is still syntactically a by block. skip almost never finishes a real goal.

The search command tries a ladder of these before it asks the model. Equation-shaped goals (a = in the statement) try ring and omega earlier. That is a heuristic, not a theorem about tactics.

Imports

import Mathlib imports the umbrella module. It is convenient and heavy.

import Mathlib.Data.Nat.Basic imports one slice.

aimath’s retriever looks at file names and declaration names under the Mathlib package. It is not a full semantic premise selector. It will miss the right import often. The fallback exists because of that.

If you know the right import, you still cannot pass it on the CLI in 0.1.x. You can put related text in the statement or domain string so retrieval has tokens to find. That is clumsy. It is the truth of this version.

Unicode versus ASCII

Lean loves $\forall$, $\exists$, $\rightarrow$, $\blacksquare$. Windows consoles sometimes hate them.

aimath accepts forall, exists, and Nat. Prefer ASCII on the command line if your terminal mangles Unicode. Inside Lean files, Unicode is fine.

If you paste $\forall$ into PowerShell and the program sees a different character, you will get a confusing Lean error. Try ASCII.

Comments and the sorry scanner

Lean line comments start with `--`.

Block comments are `/- ... -/`.

aimath's `find_banned` walks the source and ignores those comments and string literals. It still forbids the identifiers `sorry` and `admit` in live code.

Do not try to hide a hole as `s!orry` or a unicode lookalike and then call the result a theorem. The kernel is the real defense. The scanner is courtesy and a second fence.

What you do not need yet

You do not need to write macros.

You do not need to understand reducibility, universe levels, or the equation compiler.

You do not need VS Code, though the Lean extension is the best way to learn if you outgrow the CLI.

You do need to read errors. A Lean error is a gift. A model apology is not.

After this chapter

You can look at a generated file and know what each part is for. The next chapter explains the library that file is trying to speak.

Chapter 6 — Mathlib from zero, as the primary library

Mathlib is the reason aimath is heavy, slow to initialize, and worth using.

What Mathlib is

Mathlib is a collaborative library of formalized mathematics for Lean 4. It is developed in the open. It contains definitions and proofs across algebra, topology, analysis, number theory, category theory, combinatorics, and more. It changes. New lemmas appear. Names move. Toolchain pins move.

aimath does not vendor a frozen private copy inside the Python package. It asks Lake to fetch the community library when you init.

What Mathlib is not

It is not “all of mathematics.”

It is not a textbook. The lemmas are written for machines and for people who already know the mathematics.

It is not automatically the same as a paper’s theorem even when the English looks similar. Formalization chooses a precise statement. The paper may have meant a stronger or weaker one.

It is not copied into aimath.sqlite. The database stores this system’s attempts and results. Mathlib remains files on disk.

Where it lives after init

```
lean_ws/  
lakefile.toml          -- require mathlib  
lean-toolchain         -- pin matching Mathlib  
.lake/packages/mathlib -- the library itself
```

workspace_ready in the Python code is a simple check: lakefile.toml exists and .lake/packages/mathlib is a directory. If someone deletes the package folder, prove refuses. That is better than running without a library and pretending.

How search uses Mathlib

The retriever:

1. Takes tokens from your query (words of length at least 3).
2. Prefers the longest token as a needle.

3. If `rg` (ripgrep) is on `PATH`, it lists Lean files containing that literal needle.
4. If not, it walks Lean files under `Mathlib/` (skipping nested `.lake` and `.git`).
5. It extracts a declaration name if a theorem/lemma/def/abbrev/axiom line matches.
6. It returns hits with a module path such as `Mathlib.Data.Nat.Basic`.

This is name and text search, not understanding. If you search `add_zero`, you may find `Nat.add_zero`. If you search a paraphrased English sentence, you may find nothing.

`known_in_mathlib` has two extra behaviors:

- If your whole statement is a single identifier, a hit whose title is that identifier (or ends with `.identifier`) counts as known.
- If the compact form of your statement (whitespace removed) appears inside a Mathlib file, that counts as known.

So $n + 0 = n$ may not be “known” merely because a lemma’s type is $n + 0 = n$ after pretty-printing, if the file writes it differently. `forall n : Nat, n + 0 = n` may match if that exact compact string exists. This is imperfect on purpose: we would rather miss a known lemma (and try to prove it again) than mark a new wording as known when it is not there.

When a hit is known, `aimath` records status known and does not call it a new theorem. That is honesty, not failure.

Why imports hurt

Opening `import Mathlib` can use gigabytes of RAM and many seconds. That is why the host budget assumes about 3 GB per Lean worker by default. You can change `host.lean_worker_gb` if you measure a different footprint on your machine. If you set it too low, you will over-subscribe RAM and the machine will swap. Swapping feels like death. Prefer fewer workers.

Cache

`lake exe cache get` downloads precompiled `olean`s. Without the cache, the first checks compile huge dependency cones. With a cold cache, a trivial proof can take minutes. With a warm cache, it can take seconds. `Init` tries to fetch the cache. If the fetch fails, you may still have sources and a miserable first build. Re-run the `cache` command inside `lean_ws` when you have a better network.

Contributing back

`aimath` does not submit lemmas to Mathlib. If you prove something that belongs there, you do the ordinary human work: check it is not already present under another name, write a PR, follow style. This book will not walk through a Mathlib PR. The Mathlib documentation does.

After this chapter

You should treat Mathlib as the floor you stand on, not as an optional rug. The next chapter explains the talkative intern: the language model.

Chapter 7 — Language models as proposers, never as judges

This chapter explains the third piece of the machine at the level of mechanism, not marketing.

What a language model does

A modern large language model takes a list of messages (a system prompt, a user prompt) and predicts a continuation. The continuation is text. The text may look like Lean. Looking like Lean is not being accepted by Lean.

aimath sends roughly:

- A system prompt that states the trust rules: only Lean counts; local novelty is not world novelty; axioms are not ordinary theorems.
- A personality bias (more in Chapter 20 of the reading order; in this book's later personality chapter).
- A user prompt that includes the goal, imports, and sometimes a previous Lean error.

The model returns a string. aimath then tries to extract tactics or JSON from that string.

Extraction is suspicious on purpose

Models wrap answers in markdown fences. They write `{"tactics": "rfl"}`. They write prose and then a block. `textutil.extract_tactics` and `parse_json_blob` try to find a fenced block or a JSON object. If they fail, the raw text is treated as a tactic script.

That means garbage can become a by block. Lean will reject garbage. Good.

If the model writes sorry, the scanner rejects the file before Lean is even a hero. Also good.

Providers

Two families are implemented.

`openai_compatible`. HTTP POST to `{base_url}/chat/completions` (the router is careful about whether you already included `/v1` or `/chat/completions`). The body is the usual messages array. Authorization is Bearer plus the API key if one is set.

This family includes OpenAI, Groq, OpenRouter, many proxies, Ollama at `http://127.0.0.1:11434/v1`, LM Studio, and vLLM. They are not the same product. They share a shape of HTTP.

anthropic. HTTP POST to a /v1/messages style URL with x-api-key and anthropic-version: 2023-06-01. A key is required. There is no “empty key for local Anthropic” path in 0.1.x.

If provider is openai_compatible, api_key_env is non-empty, the env var is unset, and the base URL is not localhost or 127.0.0.1, aimath refuses to start the client. That saves you from sending keyless requests to a cloud and reading a useless 401 after a long Lean init.

If you use Ollama, set api_key_env to the empty string.

Temperature

Each personality has a default temperature. Higher temperature means more variety and more nonsense. The prove loop uses the personality’s temperature. You cannot set temperature on the CLI in 0.1.x. Change personality or change the code.

What the model is not told

It is not given the entire Mathlib. Retrieval injects a few module names and short hits. The model will invent the rest.

It is not given a guarantee that its previous proof almost worked. It is given a truncated error string. Truncation is bounded so prompts do not explode.

It is not given permission to skip Lean. The system prompt says so. Models ignore system prompts often. Lean does not ignore holes.

Cost and energy

Every prove attempt after typecheck is at least one model call unless you are on a path that does not need one. Swarm multiplies calls. Cloud providers charge. Local models burn electricity and GPU memory.

The host budget caps in-flight calls, not total calls. A swarm of 1000 logical agents with max_llm_inflight: 2 still makes many sequential calls. It just does not make 1000 at once.

If you are teaching a lab, forbid huge swarms. If you are paying a bill, set a personal maximum before you start.

Failure modes that look like philosophy

The model emits a correct informal proof and a wrong tactic script. Students believe the informal proof. Teach them that aimath did not accept the informal proof.

The model emits a script that works by accident on a weaker goal than you intended. Check the statement.

The model claims the problem is open and then “solves” it. The open-problem catalog may warn. You must still not believe a script.

After this chapter

You should be unable to say “the AI proved it” without adding “Lean accepted the file.” The next chapter writes the trust model as a specification.

Chapter 8 — The trust model (specification)

This chapter is the legal document of the project. Later chapters implement it. If a feature violates this chapter, the feature is wrong.

Roles

Proposer. Anything that invents text: the language model, the tactic ladder, a human typing a statement, a YAML file of axioms, a curiosity prompt.

Checker. Lean 4, invoked as lake env lean in the Mathlib workspace, plus the sorry/admit scanner.

Recorder. SQLite, reports, Lean files written under Aimath/User and Aimath/Research.

Only the checker can authorize the recorder to store status theorem.

Status contract

Status	Who may assign it	Meaning
---	---	---
theorem	Checker success, no banned tokens	This file is a proof of this statement in this system
known	Retriever + Mathlib text/name match	Already in Mathlib as we search it; not a new local theorem
conjecture	Typecheck success, proof failure or budget exhaustion	Well-formed, unproved here
rejected	Triviality, banned tokens, typecheck failure	Not a current candidate
accepted (system)	Lean accepted the axiom file	Extra axioms may be used with --system
proposed (system)	Stored but not Lean-clean	Do not prove against it
library	Built-in	The Mathlib row in systems

True and False as entire statements are trivial and rejected. They are not deep philosophy. They are a filter against a model that shrugs.

Relativity

If --system is not mathlib, a theorem must be spoken as relative to that system. The CLI prints that phrase. Reports should too. A relative theorem is a real formal object and a false advertisement if you drop the label.

Novelty contract

The function assess_novelty may say:

- known_in_mathlib
- already_in_corpus

- adjacent_to_open_problem
- literature_hits_untrusted
- new_to_this_corpus

The last one must be explained as local. The UI and this book use the sentence: not a claim of a new mathematical truth.

Open-problem adjacency is lexical. It is a seatbelt, not a classification of all open problems in the world. Absence from open_problems.json means nothing.

Untrusted sources

Hits from Wikipedia, OEIS, arXiv, notes/, foreign/, and the open-problem catalog are untrusted=True except Mathlib and the corpus's own formal rows.

Untrusted text may appear in prompts. It must not appear as a proof.

Host ethics

The default budget leaves one logical CPU core unused when the machine has more than one, and a fraction of RAM (default 25%) unused. Process priority is lowered once (below normal on Windows, nice(5) on Unix when it works). These are defaults, not a proof that you cannot still freeze a laptop if you set lean_worker_gb to 0.1 and import Mathlib twelve times.

Distribution ethics

serve on a non-loopback address requires a token. A token is a shared secret, not a full security architecture. Do not expose the port to the internet and call it “a cluster.”

Workers in 0.1.x execute check jobs. They do not become remote prove oracles unless you extend them and keep the same checker rule on every node.

What the notebook is allowed to forget

Mathlib itself. Re-download it.

What the notebook is not allowed to forget if you care about your work: aimath.sqlite, your yaml, your notes, your accepted User and Research Lean files.

Forbidden future features (unless the contract changes in public)

- A flag that stores theorem on sorry.
- A flag that treats Wikipedia as Mathlib.

- A flag that marks open problems solved because a model said so.
- Silent axiom insertion to make a homework true.

If you add those, you forked a different project. Change the name.

After this chapter

Installation can begin. You know what the files are for before you know where they sit. The next chapter is the anatomy of disk.

Chapter 9 — Anatomy of an aimath directory

After `aimath init`, a working directory is not “the Python package.” It is a small world. This chapter names every important path.

Two trees you must not confuse

The source tree is where you cloned or unpacked the aimath project: `src/aimath/`, `docs/`, `templates/lean_project/`, `pyproject.toml`. You install that tree with `pip`. You read this book from `docs/`.

The work tree is wherever you run `aimath init`. It may be the same folder. It may be another folder. It contains `aimath.yaml`, `lean_ws/`, and later `aimath.sqlite`.

If you `init` inside the source tree, `.gitignore` already ignores `lean_ws/`, `aimath.yaml`, and `*.sqlite`, because those are local state, not the program.

aimath.yaml

This is the configuration. Paths inside it are relative to the directory that contains the file, not relative to your current working directory if you passed `--config` from elsewhere. `load_config` resolves that on purpose.

If the file is missing, commands that need it tell you to run `init`. `AIMATH_CONFIG` can point at a file. `--config` can too.

`aimath init` copies `config.example.yaml` from the project if `aimath.yaml` does not exist. It does not overwrite an existing yaml unless you are thinking of `--force` (force replaces `lean_ws`, not the yaml; the CLI keeps an existing yaml even on `--force`).

lean_ws/

A Lake project created from `templates/lean_project/`:

- `lakefile.toml` — package name `aimath_ws`, requires `mathlib` from `leanprover-community`, a Lean lib named `Aimath`.
- `lean-toolchain` — overwritten at `init` from `Mathlib`’s current pin URL.
- `Aimath.lean` — an empty root module so lake build does not elaborate the universe.
- `.lake/packages/mathlib` — after a successful update.
- `.aimath_scratch/` — temporary `Job_<uuid>.lean` files. They are deleted after each check in the happy path. If a process is killed, a stray file may remain. You may delete the scratch folder.

Under `Aimath/User/` the installer of a formal system writes `Name.lean`.

Under Aimath/Research/ a kept research definition is written.

These Lean files are artifacts. Back them up if you care. They are how a later human reads what the machine kept.

aimath.sqlite

Created on first use of the corpus (prove, status if the file exists, etc.). Tables:

systems. name, lean_src, status, timestamps. One row named mathlib is ensured with status library.

statements. The claim text, a content hash of system + normalized text, status, personality, detail, foreign key to systems.

attempts. Every check: kind (prove, search, ...), the Lean source, whether it was accepted, the output (often including tactic:ring so failures can be skipped later).

timings. Recent lean and llm durations in milliseconds. Steering uses a short average of the last five.

The hash means that whitespace-only changes of a statement are the same statement. That is usually what you want.

notes/ and foreign/

These directories do not have to exist. If they do, retrieval walks them for .txt, .md, .lean, .v, .thy, .mm, .ml and returns untrusted snippets when tokens match.

notes/ is for your papers and markdown.

foreign/ is for excerpts from Coq (.v), Isabelle (.thy), Metamath (.mm), and similar. Presence of a Coq theorem does not create a Lean theorem.

reports/

Created by aimath report.

- report.md — human ranking: theorems first, then conjectures, then known, then rejected. Relative systems are labeled.
- Report.lean — concatenation of accepted Lean sources, or a comment that none exist.

These files are overwritten when you report again.

The Python package (source tree)

You will not need this until Part V, but names help:

```
| Path | Job |  
| --- | --- |  
| src/aimath/cli.py | All commands |  
| src/aimath/config.py | YAML |  
| src/aimath/textutil.py | Hash, sorry scan, JSON |  
| src/aimath/host/ | Probe, budget, sensors |  
| src/aimath/runtime/ | Protocol, scheduler, sockets |  
| src/aimath/llm/router.py | HTTP to models |  
| src/aimath/lean/ | Init + sandbox |  
| src/aimath/knowledge/ | Corpus, retrieve, sources |  
| src/aimath/agents/ | Loop, search, swarm, research, curiosity, report |  
| src/aimath/formal/ | YAML systems |
```

Environment variables

AIMATH_CONFIG — yaml path.

Whatever llm.api_key_env names — the secret. Do not commit it. Do not put the secret in yaml. yaml names the env var, not the key.

Permissions and priority

On start of a probe, aimath tries to lower its own process priority. This does not lower Lean children on all operating systems in all cases, but it is the intent: do not steal the mouse. Lean children are still heavy. Close games.

After this chapter

You can look at a folder and know whether it is a work tree. Chapter 10 walks through creating one, including the Windows minutes.

Chapter 10 — Installation, minute by minute

This chapter assumes you have never installed a proof assistant. It is long on purpose. If an expert is bored, good: a beginner can finish without a chat window.

10.1 Decide where the work tree will live

Pick a directory with tens of gigabytes free. Mathlib's cache is the large object. SSD is strongly preferred. Network filesystems and synced folders (OneDrive, Dropbox) cause mysterious locking. Prefer a local disk path such as `D:\work\aimath-work` or `~/aimath-work`.

You may init inside the cloned repository. That is convenient for developers. For a clean life, clone the repo, install the package, init in a sibling folder.

10.2 Python

You need Python 3.11 or newer.

On Windows, from PowerShell:

```
python --version
```

If that fails, install Python from python.org and tick “Add python.exe to PATH.” Open a new terminal after installing.

Create a virtual environment so aimath's dependencies do not fight other projects:

```
cd D:\work\aimath-work
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If execution policy blocks the activate script:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Then activate again. You should see `(.venv)` in the prompt.

On Unix:

```
python3 -m venv .venv
source .venv/bin/activate
```

10.3 Install the aimath package

If you have the source:

```
python -m pip install -e ".\path\to\AI_mathematician[dev]"
```

From inside the repo:

```
python -m pip install -e ".[dev]"
```

`.[dev]` adds `pytest`. `.[docs]` adds `reportlab` for the PDF book. You can write `.[dev,docs]`.

Confirm:

```
aimath --help
```

If `aimath` is not found, the `venv` is not active, or your `PATH` does not include the `venv` scripts directory. Use `python -m aimath --help` as a fallback. That form uses the current interpreter and is the more reliable teaching command.

10.4 Install elan (Lean's version manager)

`aimath` needs `lake` on `PATH`.

Windows, the way that works

Do not use `irm https://raw.githubusercontent.com/leanprover/elan/master/elan-init.ps1 | iex` if you have seen "The underlying connection was closed." Windows PowerShell's `irm` often fails that download.

Do not type `curl` unless you are sure it is `curl.exe`. In Windows PowerShell, `curl` is often an alias for `Invoke-WebRequest`, which will not save the installer the way the official instructions expect.

Type this:

```
curl.exe -L -o elan-init.ps1 https://elan.lean-lang.org/elan-init.ps1
powershell -ExecutionPolicy Bypass -File .\elan-init.ps1
Remove-Item .\elan-init.ps1
```

The installer asks questions. The defaults are sensible: install to `~\elan`, modify `PATH`. Accept the default toolchain if asked; `aimath init` will pin the project anyway.

Close the terminal and open a new one. `PATH` changes do not always apply to the window that ran the installer. Activate your `venv` again in the new window.

Check:

```
lake --version
elan --version
```

If `lake` is still unknown, the `PATH` modification failed. Add `%USERPROFILE%\elan\bin` to your user `PATH` by hand, new terminal, try again.

Other platforms

Official elan README:

```
curl https://elan.lean-lang.org/elan-init.sh -sSf | sh
```

Then open a new shell.

10.5 Optional: ripgrep

Install rg from <https://github.com/BurntSushi/ripgrep>. Without it, Mathlib search walks files. With a full Mathlib tree that walk is slower. aimath still works.

10.6 Optional: a local model

If you do not want a cloud key, install Ollama, then:

```
ollama serve  
ollama pull llama3.1
```

Wait until the model is actually pulled. Then use the yaml snippet in the next section.

LM Studio and vLLM also work if they speak the OpenAI chat completions shape.

10.7 aimath init

In the work directory, with venv active and lake visible:

```
aimath init
```

What should happen, in order:

1. aimath.yaml is written if missing (copy of the example).
2. lean_ws/ is copied from the template.
3. The toolchain pin is downloaded from lean.toolchain_url (Mathlib master pin by default).
4. lake update runs inside lean_ws.
5. lake exe cache get runs.
6. The program checks that .lake/packages/mathlib exists.

This can take a long time and a lot of bandwidth. Do not interrupt the first successful cache if you can help it. If you do interrupt, the tree may be half-written. Then aimath init --force replaces lean_ws. --force deletes that folder. Do not --force if you have precious files only in lean_ws/Aimath/ unless they are backed up.

If lake is missing, init prints the elan help and exits. It does not enter an “unverified mode.” That is a feature.

10.8 Configure the model

Edit `aimath.yaml` with a text editor.

Ollama:

```
llm:
  provider: openai_compatible
  base_url: http://127.0.0.1:11434/v1
  model: llama3.1
  api_key_env: ""
```

OpenAI:

```
llm:
  provider: openai_compatible
  base_url: https://api.openai.com/v1
  model: gpt-4o-mini
  api_key_env: OPENAI_API_KEY
```

Then in the same terminal session:

```
$env:OPENAI_API_KEY = "sk-..."
```

On Unix: `export OPENAI_API_KEY=sk-...`

The variable must be set in the terminal that runs `aimath`. Setting it in a different window does nothing.

Anthropic: provider `anthropic`, base `https://api.anthropic.com`, `api_key_env: ANTHROPIC_API_KEY`.

10.9 Confirm readiness

```
aimath status
aimath resources
```

You want `mathlib` ready: yes. You want a Lean worker count of at least 1. On a large Windows machine you may see many workers; that is the budget math, not a command to use them all at once for fun.

10.10 Common install failures

`aimath` not found. Venv, `PATH`, or `python -m aimath`.

`lake` not found after `elan`. New terminal; check `~\elan\bin`.

`irm` failed. You used the old GitHub one-liner. Use `curl.exe` and `elan.lean-lang.org`.

Init failed mid-lake. Network, proxy, disk. Fix, then `--force` if the folder is junk.

Antivirus locks files under `.lake`. Exclude the work tree, or wait, or retry.

OneDrive. Move the work tree out of a synced folder.

Python 3.10. Upgrade. The package requires 3.11+.

10.11 What you still have not done

You have not proved anything. Chapter 12 is the first hour of use. Chapter 11 explains every yaml key before you change them at random.

Chapter 11 — Configuration, key by key

The file is YAML. A tab in the wrong place will break it. Use spaces. If load fails, the message will say the file must be a mapping or a section must be a mapping. Fix the yaml before suspecting Lean.

Paths in the file are relative to the yaml's directory.

Unknown llm.provider values are rejected. Only openai_compatible and anthropic exist in 0.1.x.

Section llm

provider. openai_compatible or anthropic.

base_url. For OpenAI-compatible, typical values:

- `https://api.openai.com/v1`
- `http://127.0.0.1:11434/v1` (Ollama)
- A Groq or OpenRouter base that already ends in `/v1`

The router will append `/chat/completions` if you gave a `/v1` root. If you already gave the full chat URL, it leaves it.

For Anthropic, `https://api.anthropic.com` is enough; the router appends `/v1/messages` as needed.

model. A string the server understands. llama3.1 is not gpt-4o-mini. Wrong names produce HTTP 404 or a model-specific error. aimath surfaces the first 800 characters of the body.

api_key_env. Name of an environment variable, or empty. Empty is correct for many local servers. For Anthropic, empty will fail when the client is built.

The secret is never read from yaml. If you paste a key into yaml, you will eventually commit it. Do not.

Section host

ram_reserve_ratio. Default 0.25. Fraction of total RAM that should remain “for the rest of the machine.” If available RAM is below that reserve, the budget sets memory_pressure and collapses to one Lean worker and one LLM call.

Worked example, also in the tests: 8 logical cores, 32 GB total, 24 GB available, reserve 25%, 3 GB per Lean worker.

- Reserve = 8 GB.
- Pressure? $24 > 8$, no.

- Usable = $24 - 8 = 16$ GB.
- By RAM: $16 // 3 = 5$.
- By cores: $8 - 1 = 7$.
- Lean workers = $\min(5, 7) = 5$.
- LLM in-flight = 2 if you asked for 2.

If available is 4 GB on the same 32 GB machine, pressure is true ($4 < 8$), Lean workers = 1, LLM = 1.

If the machine has 1 core, cores_cap is 1, not 0. A single-core machine still gets one worker.

lean_worker_gb. Default 3. If usable RAM is less than one footprint, you still get one worker (so you can try), unless other steering shrinks you later. That one worker may still swap. Free memory.

max_llm_inflight. Default 2. Ignored down to 1 under pressure.

Steering after the raw budget (disk, temperature, timings) is Chapter 21 in spirit; it lives in steer_budget:

- Disk free under 5 GB: one Lean worker.
- Temperature ≥ 90 C if a sensor exists: one Lean and one LLM.
- Recent Lean ms $> 3 \times$ recent LLM ms: LLM cut to 1.
- Recent LLM much slower: Lean caps stay; a note is recorded. We do not add Lean workers beyond the host cap just because the model is slow.

GPU name is displayed. It does not start CUDA work. Lean 4's ordinary checks are CPU.

Section lean

workspace. Default lean_ws. This is where Lake lives.

timeout_seconds. Default 180. A single lake env lean beyond this is a timeout (reason includes the number of seconds). Increase for huge imports on a cold cache. Do not increase instead of fetching the cache.

toolchain_url. Default is Mathlib's lean-toolchain on GitHub master. Init downloads the first line. If you need an air-gapped pin, you can change the URL to something you host, or you can write lean-toolchain by hand after a template copy — but then you are off the paved road.

Section knowledge

arxiv. Default false in the loader. Example file may show false. When true, explore/search retrieval may fetch abstracts from export.arxiv.org. Abstracts are untrusted. Full PDFs are not downloaded in 0.1.x.

web. Example file may set true. The loader defaults false if the key is missing. When true, Wikipedia search API and OEIS JSON search are queried. Network, untrusted, never proofs.

db. Default aimath.sqlite.

notes. Default notes.

foreign. Default foreign.

Section distributed

host. Default 127.0.0.1.

port. Default 8765. serve may pass --port 0 only if you change code; the CLI uses the configured port. The coordinator binds and reports the actual port if the OS assigns one — in tests we bind port 0. The production CLI uses the yaml port.

token. Empty allowed only on loopback (127.0.0.1, localhost, ::1). Any other host without a token raises: a token is required.

Finding the file

Order: --config argument, else AIMATH_CONFIG, else ./aimath.yaml from the current working directory. There is no search up the parent chain in 0.1.x. If you run the command from the wrong folder, you will be told to init. cd first.

After this chapter

You can change a knob and know which subsystem you touched. Chapter 12 is the first session of actual mathematics-shaped commands.

Chapter 12 — The first hour of use

You have a ready workspace and a model. This chapter is a script. Type the commands. Read the commentary even if you skip typing.

Minute 0 — See the machine

```
aimath resources
```

Read every line. If disk is low, stop and free space before Mathlib checks. If GPU is “none detected,” that is normal; you do not need a GPU for Lean. If model latency is unavailable, your `base_url` host may be down (Ollama not serving) or unreachable.

```
aimath status
```

Confirm mathlib ready: yes. If no, do not continue to prove. Return to Chapter 10.

Minute 5 — A statement you already believe

```
aimath prove --statement "forall n : Nat, n + 0 = n" --personality elementary --attempts 3
```

Possible first lines of output:

known. Mathlib already has this or the compact text. You did not fail. You learned the system will not invent priority. Read the detail line; it may name a module.

theorem. Lean accepted a script. The Lean source may be printed. Save it mentally: this is what a success looks like.

conjecture. Typecheck worked; three attempts did not. Look at the detail. If it is a Lean error, the model is writing bad tactics. Try `aimath search` on the same statement.

rejected. The statement did not elaborate, or it was trivial, or it contained sorry. If you typed a curly quote instead of a straight quote in PowerShell, the statement may be wrong. Retype with straight quotes.

If the process errors with a model HTTP failure, fix `yaml` and the `env` var. Lean never ran, or ran only the typecheck.

Minute 20 — Search without trusting the first idea

```
aimath search --statement "forall n : Nat, n = n" --beam 4 --personality formula
```

This tries `rfl` early on a non-equation-looking wait: the statement has `=`, so the equation ladder actually starts with `ring`, then `omega`, and so on. `forall n : Nat, n = n` contains `=`. The ladder may close it with `rfl` later in the beam, or `ring` may already succeed, or the dummy typecheck is the slow part.

Watch the novelty paragraph. search prints `assess_novelty` text. It will tell you if this is merely local.

Minute 30 — Explore a domain

```
aimath explore --domain nat --personality curious --steps 3
```

You will see --- separated outcomes. Some may be known. Some rejected. Some conjectures. This is a successful explore. An explore that only prints brilliant English and no statuses would be a bug.

Do not raise `--steps` to 50 on a cloud key in the first hour.

Minute 40 — A formal system that is obviously a toy

```
aimath system load examples\parity.yaml
```

If you init'd outside the repo, copy `examples/parity.yaml` from the repo or recreate it:

```
name: Parity
notes: A toy extension. These axioms are assumptions.
axioms:
  - name: zero_even
    statement: " $\exists k : \text{Nat}, 0 = 2 * k$ "
```

accepted means Lean accepted the axiom file on top of Mathlib. You have not proved anything interesting about parity. You have assumed something that is, in this case, even true in ordinary math (0 is even), but it is still an axiom in the file, not a derived theorem. The system's point is the mechanism, not the depth.

Proving under `--system Parity` requires an accepted system and a non-trivial statement. True will be rejected as trivial. That is correct.

Minute 50 — Write the lab notebook out

```
aimath report
```

Open `reports/report.md` in an editor. Read it as if you were a skeptic. If it says theorem, look at `reports/Report.lean`. If the Lean looks like sorry, file a bug; that must not happen.

Minute 55 — What not to do in the first hour

Do not run `swarm --agents 10000`.

Do not type the Riemann hypothesis.

Do not `--force init` because prove was slow.

Do not paste the model's English into a homework without Lean and without disclosure.

Minute 60 — Where the book goes next

The following chapters are the same commands, slower: every flag, every branch of the prove loop, every status. If the first hour felt like magic, those chapters unmagic it. If the first hour failed, those chapters plus the failure appendix are the map.

Chapter 13 — prove in exhaustive detail

This chapter follows `Mathematician.prove` and `cmd_prove` line by line in spirit. If you change the loop, change this chapter.

Invocation

```
aimath prove --statement TEXT [--personality NAME] [--attempts N] [--system NAME]
```

Defaults: personality curious, attempts 3, system mathlib.

`--config` is a global option before the subcommand: `aimath --config D:\x\aimath.yaml prove ...`

CLI prelude

The command calls `_open_math(..., need_llm=True)`.

That function:

1. Loads config.
2. Refuses if Mathlib is not ready (elan help if lake missing).
3. Probes the host (and lowers priority).
4. Computes the raw budget.
5. Opens the corpus and records library system.
6. Senses disk, GPU, temperature, model TCP latency.
7. Reads recent timings; if both lean and llm averages exist, builds a Timings.
8. Steers the budget.
9. Builds the LLM client (required here).
10. Starts a Scheduler with a process pool for Lean and a thread pool for LLM.
11. Builds a Retriever with arxiv/web/notes/foreign flags.
12. Wraps `lean_check_job` so each check records a lean timing.
13. Wraps `client.complete` so each call records an llm timing.

If the client cannot be built, you never enter the loop.

On the way out, finally closes scheduler and corpus even if prove threw.

Exit code 0 if status is theorem or known. Otherwise 1. Conjecture is a non-zero exit. That is correct for scripts: “not proved” is a failure of the command’s hope, not a crash.

Enqueue

The first thing prove does is enqueue a `WorkItem` of kind `prove` with the statement, system, and personality. The local scheduler's backlog is a list. In 0.1.x this backlog is mostly observational: the prove loop itself is synchronous. The item exists so a future worker or a log can see the same shape. Swarm and tests assert that enqueue happened.

Rejection before Lean

The statement is stripped.

If it is empty, or `is_trivial` (`True`, `False`, trivial after compacting whitespace, case-insensitive), or `find_banned` sees `sorry/admit` in the statement itself, the corpus records rejected and the function returns. No model call.

Already recorded

If the same system + normalized text already has status `theorem` or `known`, prove returns that status with detail "already recorded in this corpus." It does not prove again. Delete the sqlite row if you truly want a redo (advanced; backup first).

Mathlib known

If system is `mathlib`, `known_in_mathlib` runs. On a hit, status `known`, detail names title and module, no model call.

Imports and typecheck

`modules_for` returns up to four Mathlib modules or `["Mathlib"]`.

If `--system` is not `mathlib`, `_system_import` requires that system's status to be `accepted` and adds `Aimath.User.<LeanIdent>`. If you pass a name that is not accepted, `ValueError` bubbles to the CLI.

Typecheck source is built. If the typecheck source somehow contains a banned token (should be rare), rejected.

The checker runs. If it fails and imports were not already `["Mathlib"]`, imports become `["Mathlib"]` and typecheck runs again. If it still fails, rejected with truncated stderr.

On success, a statement row is stored as conjecture with detail that it typechecks.

The attempt loop

`attempts` is at least 1.

Each iteration:

1. Enqueues prove or critique (if there was feedback).
2. Calls the model with `SYSTEM_PROMPT` + `personality.bias` and a user prompt that lists imports and any previous error.
3. Extracts tactics.
4. Renders a proof file.
5. If banned tokens, records an attempt failure and sets feedback asking to rewrite without sorry/admit. Continues.
6. Otherwise checks Lean. Records the attempt with the output.
7. If accepted: status theorem, detail mentions relativity if needed, returns with `lean_src`.
8. If not: feedback is the error, truncated.

If the loop ends, status stays conjecture with detail that no proof was accepted in the attempt budget.

What is stored on each attempt

`add_attempt` stores the full Lean source and a short output. Failed search attempts (other command) prefix `tactic:name` so `failed_tactics` can skip them. The prove path stores raw Lean errors. Search and prove share the attempts table.

Personalities

An unknown personality raises `ValueError` listing the known names. The CLI prints that and exits 1.

--system besides mathlib

You must have loaded and accepted the system. The proof file imports Mathlib modules plus `Aimath.User.Name`. The User file must exist from `install_system`. If you deleted the Lean file but left sqlite accepted, checks may fail with unknown import. Reload the yaml.

Performance notes

The first typecheck after init may compile a huge cone. Subsequent checks in the same process do not keep a warm Lean server: each check is a new lake env lean process. That is simple and slow. A persistent lean --server would be faster and is not 0.1.x. Timeouts of 180 seconds are for the cold case.

After this chapter

You can draw the prove flowchart on paper. Chapter 14 is the other ways to look for a proof.

Chapter 14 — search, explore, and swarm

These commands look similar in a README table. They are not the same algorithm.

search — a beam of tactics

```
aimath search --statement TEXT [--personality NAME] [--beam N] [--system NAME]
```

Defaults: personality formula, beam 4, system mathlib.

Ladder

If the statement contains the character =, the equation ladder is used:

ring, omega, norm_num, simp, linarith, rfl, aesop, decide

Otherwise:

rfl, simp, aesop, omega, decide, norm_num, ring, linarith

rank_scripts skips tactics that already failed for this statement (from attempts whose output lines start with tactic:), skips banned tokens, and de-duplicates.

If the model later proposes extra scripts, the function reserves beam slots for them so the ladder cannot fill every slot. That is why a proposed custom tactic still appears in tests when ring already failed.

Phase 1: no model

search_proof typechecks like prove (including fallback to import Mathlib). Then it tries the ladder with ask_model=False. If a script is accepted, it returns theorem and never calls the completer. The unit test test_solver_accepts_without_calling_the_model locks that behavior.

Phase 2: model scripts

If phase 1 failed, the model is asked for JSON {"scripts": ["simp", "ring"]}. Failed tactics are refreshed. The beam is rebuilt. Phase 2 runs with ask_model=True, which writes conjecture if nothing works.

Novelty print

The CLI prints assess_novelty after the search. Read it.

What search is not

It is not Aesop's tree search.

It is not a hammer that selects all Mathlib lemmas by Bayesian premise selection.

It is not learning a new neural tactic model. Memory is the sqlite list of failed tactic names.

It is still the most “prover-like” command in 0.1.x.

explore — invent candidates, then prove them thinly

```
aimath explore --domain TEXT [--personality NAME] [--steps N]
```

Defaults: curious, 5 steps.

Prompt

The model sees the domain, the personality, and a catalog of retrieval hits (Mathlib, corpus, open problems, notes, foreign, maybe web). Untrusted hits are labeled. It is told to return JSON {"conjectures": ["...", "..."]}

Filters

Each candidate is dropped if:

- we already kept enough (steps);
- the normalized text was already seen in this run;
- it is trivial;
- it contains banned tokens;
- the corpus already has it (any status) for mathlib;
- Mathlib known-match: recorded as known, counted as an outcome, not proved.

Survivors are passed to prove(..., attempts=1). One attempt each. That is why explore produces conjectures often. It is a net, not a siege.

Exit code

The CLI returns 0 if any outcomes were kept, even if none proved. “No conjectures were kept” is the failure. That is different from prove. Explore’s job is to produce candidates.

swarm — many logical agents, first proof wins

```
aimath swarm --statement TEXT [--agents N] [--personality NAME]
```

Defaults: 8 agents, seed personality curious.

Expansion

expand_agents builds N WorkItems. Personalities rotate through the sorted roster starting at the seed. Agent 0 is the seed if the seed is a known name. Agent 1 is the next name alphabetically, wrapping.

A thousand agents means a thousand queue items, not a thousand Python processes.

Execution

Items are enqueued (visible backlog).

run_swarm then runs them with a thread pool whose width is max(1, scheduler.budget.llm_inflight) from the CLI.

Each worker:

1. Calls prove with attempts=1 and that personality.
2. If not a theorem, calls prove again with switch_personality.
3. If a theorem appears, stores it as winner.

When width is 1, the loop breaks early after a win. When width is greater, pool.map may still start extra work already submitted. There is a winner check at the start of run_one to skip if someone else already won. This is best-effort, not a formal barrier. Extra model calls after a win are possible. They should be uncommon on width 1.

What to tell your grant report

“We queued 10,000 agents” is true if you passed --agents 10000.

“We ran 10,000 concurrent models” is false unless your budget said so (it will not).

“We proved it with a swarm” is true only if Lean accepted a file.

Choosing among the three

```
| Situation | Command |
| --- | --- |
| You have one goal and want cheap tactics first | search |
| You have one goal and want the model to talk, with retries | prove |
| You have a topic and want candidates | explore |
| You have one goal and want many styles in a queue | swarm |
```

Do not swarm an explore. That is a product of the sizes.

After this chapter

Chapter 15 covers curiosity, research, systems, and reports — the commands that are not “try to prove this string.”

Chapter 15 — curious, research, system, and report

curious — scheduled wandering, not a mind

```
aimath curious [--steps N] [--personality NAME]
```

For step in 0 .. steps-1:

1. `next_question(step, seed)` picks a field from a fixed cycle: algebra, combinatorics, number theory, geometry, analysis, physics, music, computation.
2. It picks a personality from the roster advanced by step.
3. If the field lexically matches an open problem (the matcher is on the field string, which is usually too short to match “Riemann hypothesis”), a warning note may appear. The note always tells the model to ask for a nearby Lean lemma and not to claim an open problem.
4. Retrieval runs on the field.
5. `explore` is called with `steps=1` and a domain string that includes the note and hint titles.

Curiosity is therefore `explore` with a rotating domain. It does not run when you are idle unless you wrap it in Task Scheduler or cron. There is no daemon in 0.1.x.

Physics and music are in the cycle to satisfy “inspiration from widely different fields.” The inspiration is a prompt. If the model emits a precise Lean proposition, good. If it emits a poem, Lean rejects it.

research — definitions that earn keep

```
aimath research [--steps N] [--personality NAME]
```

Each step asks the model for JSON:

```
{
  "name": "Double",
  "definition": "def double (n : Nat) : Nat := n + n",
  "goal": "forall n : Nat, double n = n + n",
  "tactics": "intro n; rfl"
}
```

Rules:

- definition must start with `def` .
- There must be a goal.
- Combined text must not contain banned tokens.
- The rendered file imports `Mathlib`, opens namespace `AimathResearch.Name`, includes the `def`, proves theorem `aimath_uses_def : (goal) := by tactics`, and closes the namespace.

If Lean rejects, any existing file for a failed new name is not kept; a previous kept file is not deleted just because a later step failed. The code unlinks the path if the check failed after write... actually it writes only after success for the happy path: it checks, and on failure it unlinks if the path exists. On success it writes. Read `research_step` if you change this; the test `test_research_keeps_only_a_proved_definition` requires that a failed Triple does not remain on disk.

A kept definition also stores a statement row as theorem with a detail that names the definition. That theorem is the consequence, not a claim that the definition was unknown to mathematics.

Research is how this project answers “come up with new systems” without filling the disk with unchecked axioms. Prefer research for definitions. Prefer system for explicit extra axioms you mean to study as assumptions.

system load

```
aimath system load FILE.yaml
```

Does not need an LLM (`need_llm=False`). Needs Mathlib, because the axiom file is checked with imports.

YAML shape:

```
name: Parity
notes: optional commentary
axioms:
  - name: zero_even
    statement: "∃ k : Nat, 0 = 2 * k"
```

name is required. axioms must be a non-empty list of {name, statement}.

`render_system` always starts with `import Mathlib` (or other imports if you change code). It sanitizes the name to a Lean identifier (`lean_ident`). Notes are stuffed into a block comment with `-/` broken so they cannot escape the comment.

Axioms become axiom ident : statement inside namespace `User.Name`.

Lean checks that file. On success, status accepted, file written to `lean_ws/Aimath/User/Name.lean`. On failure, status proposed, still written so you can read the error, and `--system` will refuse it.

sorry in the rendered source forces proposed with a clear reason.

system propose

```
aimath system propose [--personality NAME]
```

Asks the model for the same JSON shape `load_system_data` understands, then `install_system`. Garbage JSON becomes rejected with a parse message. A well-formed but Lean-failing system is proposed.

Treat proposed axioms as fiction until accepted, and as assumptions even after accepted.

report

`aimath report`

Does not need Mathlib or an LLM. Needs a config so it knows where the sqlite and the work root are.

Ranking key: theorem (0), conjecture (1), known (2), rejected (3), then shorter statements first. The human report is for attention, not for mathematical importance. A short silly theorem outranks a long deep conjecture. That is a UI choice. Change `rank_records` if you hate it, but then say so in this chapter.

`Report.lean` includes only accepted sources from attempts. If you have a theorem row without a stored accepted attempt (should be unusual), the export may skip it.

Overwrite behavior: every report replaces the two files.

After this chapter

You have the full command surface except distribution. Chapter 16 is `serve/worker`. Chapter 17 is the status words as a contract you can memorize.

Chapter 16 — serve, worker, and the JSON-line protocol

Distribution in 0.1.x is real and small. It shares Lean check jobs, not a magical remote mathematician.

Why this exists

The original request asked for concurrent, parallel, and distributed systems. The first plan deferred a cluster. The later work added a coordinator and a worker that speak the same `WorkItem` / `WorkResult` objects the local scheduler already used.

You can run the coordinator on one machine and a worker on another if you accept the security model (a shared token, no TLS in 0.1.x). The default is localhost.

aimath serve

```
aimath serve [--host HOST] [--port PORT] [--token TOKEN]
```

Unspecified flags come from `yaml distributed`.

If host is not loopback and token is empty, the process exits with a message. It does not bind first and warn later.

The coordinator:

1. Binds a TCP socket, `SO_REUSEADDR`.
2. Listens.
3. Accepts in a daemon thread.
4. Each client is another thread.

It then sleeps until `Ctrl+C`. There is no job-injection CLI in 0.1.x: the in-memory `JobQueue` starts empty unless you write a program that calls `Coordinator` and `queue.put`. The tests do that. The production CLI as shipped is the socket shape and a process you can keep up while you develop a producer.

That limitation must be stated: `aimath serve` alone does not drain your sqlite prove queue. It is the protocol endpoint. A later version should attach the scheduler backlog to this queue. Until then, treat `serve/worker` as the supported way to implement remote checks, and use the tests as the usage sample.

If you need a mental model: the book still documents the protocol completely so an extender can hook `queue.put(WorkItem(kind="check", ...))` from their own script today.

Protocol, byte by byte

UTF-8, one JSON object per line, newline at the end of each object.

Hello. Client sends `{"op":"hello","token":"..."}`. Server replies `{"op":"ok"}` or `{"op":"denied"}`.

Pull. Client sends `{"op":"pull"}`. Server sends `{"op":"empty"}` or `{"op":"work","item": "<string>"}` where `<string>` is `WorkItem.to_json()` embedded as a JSON string, not as a nested object. The worker does `WorkItem.from_json(msg["item"])`.

Result. Client sends `{"op":"result","result": "<WorkResult json string>"}`.

Bye. `{"op":"bye"}`.

If the client dies while a job is inflight, abandon requeues until `max_tries` (3), then stores a failed `WorkResult` explaining the retry budget.

WorkItem

Fields: `id` (string), `kind` (one of `prove`, `conjecture`, `check`, `critique`, `formalize`, `solve`, `search`, `research`), `payload` (object), `budget_ms` (int).

Unknown kinds raise on construction. That is a compatibility fence.

WorkResult

Fields: `id`, `ok`, `lean_accepted` (bool or null), `artifact` (object). For checks, `artifact` is often the `CheckResult` dict: `accepted`, `returncode`, `stdout`, `stderr`, `reason`.

aimath worker

```
aimath worker [--host HOST] [--port PORT] [--token TOKEN]
```

Defaults: 127.0.0.1, 8765, empty token.

The CLI worker loops: `worker_once` with `_remote_check`. `_remote_check` only handles `kind == "check"`. Other kinds return a failed result explaining the mismatch.

The payload it expects: `source` (Lean text), `workspace` (path string), `timeout` (seconds, default 180). It calls `lean_check_job` on that machine. The worker machine must have the `workspace` and `Mathlib`. A worker without `lean_ws` cannot help.

On empty, it sleeps one second and pulls again. Ctrl+C exits.

Security, said as a grown-up

- Loopback without a token is for a single-user machine.

- A token on a LAN is a shared password sent in JSON without TLS. Anyone who can read the network can read the token and the Lean sources.
- Binding 0.0.0.0 on a public IP is a bad idea.
- The worker runs Lean on whatever source the coordinator sends. That is arbitrary-code-adjacent in the sense that Lean elaborators and tactics are powerful. Only send jobs you trust. Only accept coordinators you trust.

After this chapter

You know what “distributed” means here: a socket, a queue, a check. Chapter 17 memorizes statuses. Chapter 18 is novelty language.

Chapter 17 — Status words as a contract

These words appear in CLI output, sqlite, and reports. They are not mood.

theorem

Lean accepted a proof file. The scanner found no live sorry or admit. The statement is stored with this status.

If the system is not mathlib, speak and write: theorem relative to system S.

A theorem can be trivial in the ordinary sense (for example $n = n$) and still be a theorem in the formal sense. The triviality filter only rejects the exact compact strings true, false, and trivial, not all easy lemmas.

A theorem can be uninteresting. The software does not measure interest.

A theorem can already be known to the world's mathematicians. The software does not measure that unless Mathlib search said known first. If search missed, you may have a local theorem that is an old fact. That is why reports exist for humans.

known

Mathlib search believes this is already in the library: identifier match or compact verbatim match. The prove/search loops do not try to one-up Mathlib. They record known and move on.

known is a success of retrieval. Exit code of prove is 0.

If you disagree with a known verdict, the matcher may have been too eager (verbatim compact string appeared in a comment). File a careful bug. Do not “just call it a theorem anyway” by editing sqlite.

conjecture

The statement elaborated as a Prop. No accepted proof is stored. Attempts may exist and they failed.

A conjecture in this corpus is not the Collatz conjecture. It is an open row. It may be false. Lean did not say it is true. It also did not say it is false (we do not search for disproofs as a first-class command).

explore and curious produce many conjectures. That is their job.

rejected

One of:

- empty statement;
- trivial statement;
- banned tokens in the statement or in the typecheck wrapper;
- typecheck failed (not a Prop in this environment, unknown identifiers, parse error);
- research/system parse failure (different command, similar idea: we will not keep garbage).

rejected is a kindness. It stops you from spending money on a sentence that is not a claim.

System statuses

library. The built-in Mathlib row.

accepted. Extra axiom file checked. --system Name allowed.

proposed. Stored, not allowed for prove.

Attempt rows are not statuses of statements

An attempt can be accepted=false while the statement remains conjecture. Many attempts, one statement. The statement status changes when the loop decides (set_status).

Printing rules

_print_outcome prints the status word (or the relative phrase), then the detail, then the Lean source if status is theorem and source exists.

Do not add adjectives in wrappers (“amazing theorem”). The word is enough.

After this chapter

You can read a report without translating. Chapter 18 is what you may say to other humans.

Chapter 18 — Novelty, literature, and what you may claim

This chapter is ethics with a function name.

The temptation

A model writes a file. Lean accepts it. You feel the history of mathematics move. Almost always, it did not. You formalized a lemma Mathlib search missed, or a tautology, or a restatement.

The software cannot know the history of mathematics. It can know:

- whether a compact string or identifier showed up in the local Mathlib tree;
- whether this sqlite already has the row;
- whether the text names a catalogued famous problem;
- whether untrusted sources mentioned related titles.

That is all. `assess_novelty` turns those bits into a verdict and a mandatory explanatory sentence.

Verdicts, at length

`known_in_mathlib`. Stop. Do not tweet discovery.

`already_in_corpus`. You are repeating yourself.

`adjacent_to_open_problem`. The matcher found enough tokens of a catalog name (a long token of length ≥ 6 , or two shorter name tokens). The catalog includes Riemann hypothesis, Goldbach, Collatz, twin primes, BSD, Hodge, Navier–Stokes regularity, Yang–Mills mass gap, P versus NP, Hadwiger, Jacobian, Sendov. The list is incomplete on purpose and dangerous if treated as complete. The summary says a tactic script is not a solution.

`literature_hits_untrusted`. Wikipedia or notes or arXiv mentioned something. Useful for reading. Not a proof. Not a priority search.

`new_to_this_corpus`. The honest leftover. The summary must include that this is not a claim of a new mathematical truth.

How matching works (so you can distrust it)

`match_open_problem` tokenizes the query at length ≥ 4 . It compares to tokens of the problem name. “a note on the Riemann hypothesis” hits because `riemann` is long and present. “prime” alone should not hit twin primes. If it does, tighten the matcher and add a test.

Mathlib known-matching is similarly mechanical. It will both miss and over-fire. Humans remain the literature review.

Sources you can turn on

notes/ — your files.

foreign/ — other provers' text.

arxiv: true — abstracts only, export.arxiv.org.

web: true — Wikipedia `action=query&list=search`, OEIS `fmt=json`. Need network. Failures are swallowed in `Retriever.search` so a down encyclopedia does not crash `explore`.

None of these become theorems.

Sentences you may use

Allowed:

> Lean accepted this proof in our Mathlib workspace. We did not find the compact statement in our Mathlib checkout or in our local corpus.

> This is a theorem relative to the axiom system Parity.

> The model proposed a proof of a statement that names the Riemann hypothesis. We treated that as adjacency to an open problem, not as a solution.

Forbidden:

> We solved an open problem.

> This is new to mathematics.

> Wikipedia confirms the proof.

> The AI is sure.

Coursework and publication

Disclose the tool. Your institution's rules win. A Lean file is still machine-assisted work if the tactics came from a model. Understanding the file is the part that might be yours.

After this chapter

Chapter 19 is personalities: the costumes the proposer wears, not extra authority.

Chapter 19 — Personalities, styles, and curiosity again

A personality is a name, a temperature, and a paragraph of bias. That is the entire object in `personalities.py`.

The list, with what the bias is for

curious (0.8). Simple statements near Mathlib definitions.

nerd (0.7). Definitions, edge cases, exact names.

elementary (0.4). Nat, Int, inequalities, direct tactics.

structural (0.5). Homomorphisms, identities, wide classes.

formula (0.6). Explicit identities; calc, ring, linarith, omega, simp.

cross-field (0.9). One analogy, then a precise Lean proposition. High temperature: more chaos.

euler (0.6). Examples, then an identity, then a proof. Not Euler.

noether (0.4). Invariants, structure. Not Noether.

erdos (0.5). Elementary counting. Not Erdős.

grothendieck (0.5). Maps, universal properties. Not Grothendieck.

ramanujan (0.8). Explicit numeric identities. Not Ramanujan.

The historical names are a pedagogical risk. They make demos charming and claims sloppy. This book forbids “Ramanujan found” in any output you show a human unless you mean the human Ramanujan.

Switching

`switch_personality` moves to the next name in sorted order, wrapping. Swarm uses this after a failed first prove. It is not a debate between agents. It is a second prompt.

`roster(seed, count)` is a cycle of names of length count. Unknown seed starts at index 0 (cross-field is first alphabetically, then curious, elementary, erdos, ...). If you seed curious, agent 0 is curious.

System prompt (shared)

Every completion is prefixed with a system paragraph that restates: Lean judges; local novelty is local; invented axioms are a formal system. Personalities cannot delete that paragraph. Models can ignore it. Lean cannot.

Curiosity fields

FIELDS = algebra, combinatorics, number theory, geometry, analysis, physics, music, computation.

field_for_step(n) is FIELDS[n % 8].

This is the implementation of “widely different fields.” It is a wheel, not a research program. To add a field, edit the tuple, add tests, and say in this chapter what you added.

After this chapter

Chapter 20 is the host budget as arithmetic you can reproduce.

Chapter 20 — Host awareness, budgets, and sensors

“Non-invasive” is a promise with a formula. Here is the formula.

Probe

`probe()` uses `psutil`:

- logical CPU count (at least 1);
- total RAM;
- available RAM;
- load: `getloadavg()[0]` if it exists, else `cpu_percent * cores` as a crude stand-in (Windows has no Unix load average).

It also calls `lower_priority()` once per process. Windows: `BELOW_NORMAL_PRIORITY_CLASS`. Unix: `os.nice(5)`. Failures return "unchanged" and do not crash.

Raw budget

See Chapter 11 for the worked 8-core 32 GB example. Tests in `tests/test_budget.py` lock those numbers. If you change the formula, change the tests and this chapter together.

Remember: `max(1, ...)` means we never compute zero workers. A machine that should not run Lean will still try one worker and then swap. The operator's job is to not start prove on a starving machine.

Steering

`sense_host` adds:

- GPU name from `nvidia-smi --query-gpu=name --format=csv,noheader` if the binary exists and returns 0;
- disk free on the work root;
- hottest temperature from `psutil.sensors_temperatures` if present (often missing on Windows);
- TCP connect latency to the model host (1.5 s timeout).

`steer_budget` then may shrink. It never raises workers above the raw budget. It does not start GPU Lean. The GPU line is information for humans who run local models on the same card: if the card is already full of Ollama, lower `max_llm_inflight` yourself.

Timings

Each Lean check and each LLM call records milliseconds in sqlite. recent_timing averages the last five. Steering uses that pair if both exist. A first run has no pair; steering skips the timing clause.

If Lean is more than three times slower than the model, incoming proposals are limited (llm=1) so you do not pile scripts on a saturated checker. If the model is the slow side, Lean worker count stays at the host cap; we do not add checkers you did not already pay for in RAM.

Scheduler

Scheduler holds:

- a list backlog of WorkItem;
- a ProcessPoolExecutor with spawn (Windows-safe) of size lean_workers, unless tests pass use_processes=False;
- a ThreadPoolExecutor of size llm_inflight.

lean_check_job is picklable: workspace path string, source string, timeout int, result dict.

Do not put a live sqlite connection in the process pool. The parent records timings after .result().

What we still do not sense

Fan speed, battery, whether the user is presenting, whether a video call is on. The original “optimal use” request asked for more. This is the implemented subset. If you add a sensor, fail soft: missing data must not crash prove.

After this chapter

Chapter 21 walks the source tree as an extender. Chapter 22 is people. Then limits.

Chapter 21 — Source architecture, module by module

This chapter is the map of `src/aimath/`. Read it before changing code.

Entry

`python -m aimath` → `__main__.py` → `cli.main`.

`main` calls `freeze_support()` so Windows spawn works, touches `template_dir()` so a broken install fails immediately, parses `argv`, dispatches.

config.py

Dataclasses: `LLMConfig`, `HostConfig`, `LeanConfig`, `KnowledgeConfig`, `DistributedConfig`, `AppConfig`.

`load_config` is strict about provider names and section types. Defaults apply per-key if a section exists but a key is missing. Missing sections become `{}`.

textutil.py

`normalize` collapses whitespace. `compact` removes it. `content_hash` is sha256 of `system` + `newline` + `normalize(text)`.

`find_banned` is a small lexer: skip `--` lines, `/- -/` blocks, strings with backslash escapes; otherwise read identifiers. Only exact `sorry` and `admit`.

`extract_fenced`, `parse_json_blob`, `extract_tactics`, `parse_conjectures` are the model-output parsers. They are allowed to be imperfect. Lean is the backstop.

host/

`probe.py`, `budget.py`, `sense.py` — Chapter 20.

runtime/

`protocol.py` — `WorkItem/WorkResult`, kinds tuple.

`scheduler.py` — pools.

`distributed.py` — `JobQueue`, `Coordinator`, `require_token`, `worker_once`.

llm/router.py

OpenAICompatible, Anthropic, build_client, LLMError. Timeouts default 120 seconds for HTTP, independent of lean.timeout_seconds.

lean/

project.py — ELAN_HELP, template_dir (walks parents to find templates/lean_project), lake_executable, workspace_ready, fetch_toolchain, init_workspace.

sandbox.py — CheckResult, render_imports, render_proof, render_typecheck, check_source, lean_check_job. Output clipped at 16000 characters. Scratch files deleted in finally.

knowledge/

corpus.py — sqlite, thread lock, schema, all queries.

retrieve.py — Mathlib walk, rg, hits, Retriever.search order.

sources.py — open problems, folders, Wikipedia/OEIS parsers, assess_novelty.

open_problems.json — the catalog.

agents/

personalities.py — the table and roster.

loop.py — Mathematician, prove/explore/install/propose.

search.py — beam.

swarm.py — expand and run.

research.py — definitions.

curiosity.py — wheel.

report.py — markdown and Lean export.

formal/system.py

YAML load, lean_ident, render_system, write_system_file.

tests

Tests must not download Mathlib. `test_live_mathlib_accepts_a_trivial_proof` is skipped unless `lean_ws` is ready. New tests should follow that rule.

When you add a knowledge source, add a fixture parser test. When you add a status path, add a `FakeRetriever` test. When you change budget math, change `test_budget.py` first.

Invariants to keep

1. No theorem without Lean + scanner.
2. Mathlib remains required for prove-like commands.
3. Untrusted hits stay untrusted.
4. Relative systems stay labeled.
5. Host caps remain caps.

After this chapter

You can extend without wandering. Chapter 22 tells different humans how to use the same machine.

Chapter 22 — The same system, many readers

The pamphlets under docs/audiences/ are shorter invitations. This chapter is the book's own advice, in one place, so the PDF is not a pile of disconnected flyers.

If you are in high school or just curious

Your job is to learn the four status words and to run one prove on forall $n : \text{Nat}$, $n + 0 = n$. Do not type famous unsolved problems. Do not believe English. Open reports/report.md and read it coldly. Then stop or continue as a student.

The machine is a strict grader plus a talkative intern. You remain the learner.

If you are a student (any age)

Two-week path:

Days 1–2: install, statuses, Chapter 2–8.

Days 3–5: ten prove attempts, compare elementary vs formula, use search when prove fails.

Days 6–8: explore on a small domain, notebook of prompts that worked.

Days 9–11: load parity.yaml, write “relative” in your notes.

Days 12–14: report, one page on what you still do not understand.

Disclose AI use if your course requires it. Re-prove accepted tactics by hand until you can explain each line.

If you are an undergraduate

Map courses to commands: discrete math $\rightarrow$ elementary prove; algebra $\rightarrow$ structural; analysis $\rightarrow$ expect pain (types). Do not add axioms to finish homework. known is not a bad grade; it is Mathlib.

A good project: formalize one textbook section about Nat, export a report, write which lemmas were already in Mathlib.

If you are a PhD student or working mathematician

Use notes/ and weekly report triage. Write domain strings that are narrow. Cap swarm by budget, not by enthusiasm. Do a real literature search before any public claim. Use YAML systems only when you mean extra axioms. Label talks.

The tool is an amanuensis. You still choose the question and the significance.

If you maintain a lab image

Pre-run init so class is not a download. Prefer a local model. Raise `ram_reserve_ratio` on shared boxes. Backup yaml and sqlite. Exclude `.lake` from backup if you can re-fetch cache. Collect status output when students file tickets.

If you teach

Grade explanations, not theorem counts. Pre-warm Mathlib. Forbid huge swarms. Show a live rejection of sorry. Ask “why is known not a discovery?”

If you extend the code

Read Chapter 8 and Chapter 21 first. Add tests that do not need Mathlib. Document the flag. Mark trust. Never add a shortcut around the kernel.

If you think of yourself as a master of such systems

Preserve the invariants even when a demo would look better without them. Logical scale is not process scale. Local novelty is not priority. Relative theorems stay relative. The next architecture (embeddings, hammers, durable queues) should sit on those invariants, not replace them with fluency.

After this chapter

Chapter 23 is maintenance and extension recipes. Chapter 24 is limits. Chapter 25 closes.

Chapter 23 — Maintenance recipes and extension recipes

Weekly operations

1. aimath status and aimath resources.
2. Copy aimath.sqlite and aimath.yaml to a backup folder dated today.
3. If you care about artifacts, copy lean_ws/Aimath/ and reports/.
4. Glance at disk free. Under 5 GB, steering will already punish you; free space anyway.

Monthly operations

1. Read reports/report.md. Drop conjectures you will not pursue (advanced: delete rows; or leave them; they are cheap).
2. Update the Python package (pip install -e ".[dev]") if the source changed.
3. Run pytest.

Upgrading Mathlib

This is deliberate.

1. Backup sqlite, yaml, Aimath/.
2. aimath init --force only if you accept replacing lean_ws.
3. Wait for cache.
4. aimath status.
5. A trivial prove or the live test.

Do not upgrade in the middle of a student lab.

Adding a personality

1. One entry in PERSONALITIES.
2. A test that get_personality works.
3. A sentence in Chapter 19 and the glossary.

Adding a knowledge source

1. Parser + fixture test in knowledge/sources.py style.
2. Config flag.

3. Hook in `Retriever.search`, untrusted unless it is Mathlib.
4. Manual + Chapter 18.
5. Network failures must not crash search.

Adding a CLI command

1. Function with a clear `need_ltm` / need Mathlib story.
2. `cmd_*` and subparser.
3. Tests for the parser or the loop with fakes.
4. A chapter or a section in this book.

Adding a work kind

1. `WORK_KINDS`.
2. Worker handler if remote.
3. Protocol chapter.
4. Tests for JSON round-trip.

Adding a remote prove

Not in 0.1.x. If you add it: the worker must run the same scanner and the same sandbox. Do not trust a worker that returns `lean_accepted: true` without a file. Prefer sending the source back in the artifact.

Documentation obligation

If user-visible behavior changes and this book is silent, the book is wrong. Update the numbered chapter, not only the README.

After this chapter

Chapter 24 names the gaps so they cannot hide inside enthusiasm.

Chapter 24 — Limits, absences, and the original request

The first plan contained a section “Not in this build.” This chapter is that section, written as part of the book so it cannot be skipped.

New mathematical truths (the strong sense)

We store Lean-checked theorems new to this corpus. We do not have a complete literature search, a living open-problem database with formal statements, or a standard for claiming priority. Chapter 18 is the substitute: language discipline.

A repository of all mathematics

Mathlib is primary. We optionally fetch abstracts and search local folders. We do not ingest encyclopedias in full, OEIS as a formal library, textbooks, or other provers as checkers. Isabelle, Coq, Metamath are hint directories.

A prover at prover scale

We have a ladder and a beam and a one-attempt explore. We do not have best-first proof search, a Mathlib hammer, trained premise selection, or a CAS-style solver distinct from proving.

Thousands or millions of live agents

We have a queue. Live work is capped by RAM and cores. Saying “a million agents” without saying “logical” is a lie.

A research program that invents theories

research keeps a definition if a consequence checks. system propose can emit axioms. We do not have an automatic theory builder that retains useful notation over weeks.

Optimal host use

We have CPU, RAM, reserve, priority, Mathlib footprint, disk, optional temperature, optional GPU name, optional TCP latency, and timing steer. We do not have thermal policies beyond 90 C, network-quality routing, or automatic “run Lean on the other box because it is faster.”

Full distribution

We have a protocol, requeue, token on non-loopback. We do not have TLS, durable queues, authentication beyond a shared string, or a CLI that feeds prove jobs into serve for you.

Rich inner lives

We have prompt styles and a field wheel. We do not have collaborating agents with memory of each other's sketches beyond sqlite attempts, and we do not have a daemon.

A body of work as papers

We have report.md and Report.lean. We do not have LaTeX papers, journal style, or a human review UI.

Why list absences

Because a comprehensive book that only celebrates features trains the reader to hallucinate the rest. The software is already useful if you wanted a Lean-checked loop with a host budget and an honest notebook. It is not the completed Leibniz machine. The mapping in Chapter 4 still stands.

After this chapter

The last chapter is short. The appendices are for lookup.

Chapter 25 — Afterword

A language model can sound like a mathematician. Lean can check whether a file is a proof. Mathlib can be the floor. A notebook can remember what this machine has seen. None of that is a soul, a department, or a claim of priority.

If this book did its job, you can install the system without folklore, you can choose a command without treating them as synonyms, you can read a status word without translating it into a press release, and you can change the code without breaking the only rule that makes the project worth having:

The model proposes. Lean decides. Mathlib anchors. You remain responsible for meaning.

When you are tired of chapters, run `aimath report` and read the markdown as a stranger. If you would not sign it, do not sign it.

That refusal is the feature.

Appendix A — Failure encyclopedia

Install and environment

aimath is not recognized. The virtual environment is inactive, or scripts are not on PATH. Activate the venv. Use `python -m aimath`.

lake is not recognized. elan is missing or this terminal is older than the install. New terminal. Check `~/elan/bin`. Windows: use `curl.exe` and <https://elan.lean-lang.org/elan-init.ps1>, not `irm` on GitHub raw.

`irm`: underlying connection was closed. Expected on some Windows PowerShell + GitHub raw combinations. Do not retry ten times. Change method.

`curl` created a file named `--location`. You used the PowerShell alias. Use `curl.exe`.

`init`: workspace already exists. Delete it only if you mean to, or use `--force` knowing it deletes `lean_ws`.

`init`: lake failed. Network, proxy, disk, antivirus. Read the `+ lake ...` line. Fix the environment, retry.

`status`: mathlib ready: no. `.lake/packages/mathlib` is missing. `Init` is not finished.

Configuration and models

unknown `llm.provider`. Only `openai_compatible` and `anthropic`.

environment variable `X` is not set. Cloud endpoint with a named key. Export the var in this terminal, or switch to localhost and clear `api_key_env`.

Anthropic requires an API key. You chose `anthropic` with an empty key.

model returned 401/403/404/429. Key, model name, quota. The body is clipped to 800 characters in the exception.

model request failed. Connection refused: Ollama not running, wrong port, firewall.

this command needs a configured language model. A path used `need_llm=True` and you still somehow had no client — or you called a completer in a no-llm context.

Proving

rejected: empty, trivial, or uses `sorry/admit`. Change the statement.

rejected: Lean errors about unknown identifier. Missing import or wrong name. Try a smaller statement. Put tokens Mathlib search can find.

rejected: type mismatch. The sentence is not a Prop as written.

conjecture after prove. Attempts exhausted. Try search, more attempts, another personality, or a smaller goal.

known. Not a failure.

timeout after 180s. Cold cache or huge import. Fetch cache. Narrow imports. Increase timeout_seconds only after that.

sorry is not a proof. The model (or you) used a hole. Good rejection.

Swarm and money

Many sequential model calls with a small in-flight cap. Stop the process. Lower --agents.

Distribution

a token is required when the coordinator is not bound to localhost. Set a token.

coordinator rejected the token. Mismatch.

this worker only runs check. You queued a prove item to a 0.1.x CLI worker.

job requeued then dropped. Worker died three times. Fix the worker machine's Lean.

Database

Locked sqlite if you copy the file while a command runs. Close aimath, then copy.

Corrupt yaml: tabs, bad indentation. Validate with a yaml linter.

Unicode

PowerShell ate $\forall$. Use forall.

Appendix B — Glossary (long)

`admit`. Lean hole; banned like `sorry`.

`aimath`. This project's command and Python package.

`aesop`. Automation tactic that may appear on the search ladder.

`api_key_env`. yaml key naming an environment variable that holds a secret.

`artifact`. Free-form object on a `WorkResult`.

`arxiv` (flag). When true, fetch untrusted abstracts.

`available RAM`. `psutil`'s available memory, used for pressure and usable bytes.

`axiom`. An assumed starting sentence; not a theorem.

`backlog`. List of `WorkItems` on the Scheduler.

`banned tokens`. `sorry` and `admit` as live identifiers.

`base_url`. Model HTTP root.

`beam`. How many tactic scripts search will try in a ranked list.

`budget`. Caps: `lean_workers`, `llm_inflight`, `reserve`, `pressure`.

`budget_ms`. Soft time field on a `WorkItem` (timeouts actually use `lean.timeout_seconds` for checks).

`cache` (Lake). Prebuilt oleans from lake exe cache get.

`characteristica universalis`. Leibniz's hoped-for precise language; mapped to Lean, not equal to Lean.

`check` (kind). A job that runs Lean on a source string.

`CheckResult`. `accepted`, `returncode`, `stdout`, `stderr`, `reason`.

`compact`. String with all whitespace removed; used in verbatim Mathlib match.

`conjecture`. Typechecks; unproved in this corpus.

`content_hash`. sha256 of system plus normalized statement.

coordinator. aimath serve process.

corpus. SQLite notebook.

cross-field. High-temperature personality that asks for one analogy made precise.

curious (command). Rotating-field explore.

curious (personality). Default exploratory style.

definition. A def; names a thing.

distributed. yaml section for host/port/token.

elan. Lean version manager.

elementary. Personality for Nat/Int style goals.

elaborator. Lean front-end that produces a kernel term.

equation ladder. Tactic order used when = appears in the statement.

explore. Propose conjectures and prove each with one attempt.

foreign/. Other-prover excerpts; untrusted.

formal system. Extra axioms on Mathlib; results relative to it.

formula. Personality that prefers identities and ring/omega.

forall. ASCII universal quantifier.

GPU name. Display-only sensor from nvidia-smi.

hash. See content_hash.

host. yaml section for reserves and caps.

import Mathlib. Convenient, heavy umbrella import.

init. Create yaml and Mathlib workspace.

in-flight. Concurrent live LLM calls, not total calls.

kernel. Lean's trusted type checker.

known. Mathlib hit; not a new local theorem.

Lake. Lean build tool.

lake env lean. How aimath invokes the checker.

lean_ident. Sanitized Lean identifier from a YAML name.

lean_worker_gb. Assumed RAM per Mathlib-importing process.

library (status). The Mathlib system row.

linarith. Inequality tactic on the ladder.

LLM. Language model; proposer.

llm_inflight. Cap on concurrent model calls.

load average. Unix load; Windows uses a CPU-percent stand-in.

logical agent. A queue item, not necessarily a process.

loopback. 127.0.0.1, localhost, ::1.

Mathlib. Primary formal library.

memory_pressure. available RAM below reserve.

module. Lean import path such as Mathlib.Data.Nat.Basic.

Nat. Lean natural numbers.

new_to_this_corpus. Novelty leftover; not world novelty.

nice. Unix priority lowering.

norm_num. Numeric tactic.

notes/. Local untrusted documents.

novelty. Local comparison to Mathlib, corpus, catalog, hints.

omega. Linear arithmetic tactic.

open_problems.json. Famous-problem warning catalog.

Or.inr trivial. Dummy used to typecheck a Prop without proving it.

personality. Name + temperature + bias paragraph.

proposed (system). Not Lean-clean; cannot --system.

Prop. Type of propositions.

prove. Typecheck then ask for tactics until attempts or success.

provider. openai_compatible or anthropic.

rfl. Reflexivity tactic.

relative theorem. Proved using extra axioms.

report. Write reports/report.md and Report.lean.

research. Keep a def only if a consequence checks.

reserve. ram_reserve_ratio times total RAM.

Retriever. Search object: Mathlib, corpus, extras.

rg / ripgrep. Fast file search if installed.

ring. Algebraic identity tactic.

roster. Cyclic list of personality names.

sandbox. Temp file + lake env lean + scanner.

scheduler. Process pool + thread pool.

search. Ladder then model beam.

simp. Simplifier tactic.

sorry. Hole; banned.

spawn. Multiprocessing start method; required on Windows.

statement. A proposition string you asked about.

status (command). Readiness and counts.

steer_budget. Shrink caps using disk, heat, timings.

swarm. Many logical prove agents; first theorem wins.

system load/propose. Extra axiom theories.

tactic. Instruction in a by block.

temperature. Model randomness; set per personality.

theorem. Kernel-accepted proof in this corpus.

timeout_seconds. Per-check wall clock.

token (distributed). Shared secret off-loopback.

toolchain. lean-toolchain pin.

trivial (filter). True/False/trivial compact strings.

typecheck dummy. $(P) \vee \text{True}$ proved by Or.inr trivial.

untrusted. Hint source; never a proof.

User.Name. Namespace for loaded axioms.

web (flag). Wikipedia + OEIS.

work tree. Directory with aimath.yaml after init.

WorkItem / WorkResult. JSON protocol objects.

worker. Pulls check jobs.

workspace_ready. lakefile + mathlib package dir.

Appendix C — Command, config, and ladder tables

Commands

Command	Mathlib	LLM	Purpose
init	downloads	no	yaml + lean_ws + cache
resources	no	no	probe + sensors + caps
status	checks	no	readiness + counts
prove	yes	yes	typecheck + tactic attempts
explore	yes	yes	invent + prove(attempts=1)
search	yes	maybe	ladder then beam
swarm	yes	yes	N logical agents
research	yes	yes	keep def iff consequence
curious	yes	yes	rotate field, explore 1
system load	yes	no	YAML axioms
system propose	yes	yes	model axioms
report	no	no	markdown + Lean export
serve	no	no	bind protocol
worker	yes for checks	no	pull checks

Global: --config PATH. Env: AIMATH_CONFIG.

prove flags

--statement required. --personality default curious. --attempts default 3. --system default mathlib.

search flags

--statement required. --personality default formula. --beam default 4. --system default mathlib.

swarm flags

--statement required. --agents default 8. --personality default curious.

Equation ladder

ring, omega, norm_num, simp, linarith, rfl, aesop, decide

General ladder

rfl, simp, aesop, omega, decide, norm_num, ring, linarith

Personalities (sorted)

cross-field, curious, elementary, erdos, euler, formula, grothendieck, nerd, noether, ramanujan, structural

Curiosity fields

algebra, combinatorics, number theory, geometry, analysis, physics, music, computation

Work kinds

prove, conjecture, check, critique, formalize, solve, search, research

Novelty verdicts

known_in_mathlib, already_in_corpus, adjacent_to_open_problem, literature_hits_untrusted,
new_to_this_corpus

Budget sketch

```
cores_cap = 1 if cpu==1 else cpu-1
reserve = total_ram * ram_reserve_ratio
pressure = available < reserve
usable = max(0, available - reserve)
by_ram = 1 if pressure or usable < footprint else usable // footprint
lean_workers = max(1, min(cores_cap, by_ram))
llm_inflight = 1 if pressure else max(1, max_llm_inflight)
```

Then steer: disk < 5GB → lean=1; temp≥90 → lean=1,llm=1; lean_ms > 3*llm_ms → llm=1.

Appendix D — How to write a --statement (pedantic)

Most failed runs are not mysterious. The sentence you passed is not the sentence you meant, or it is not a Lean Prop. This appendix is a drill.

Start from English, then refuse English

Write the claim in English on paper.

> Adding zero does nothing.

That is a slogan. It has no quantifier and no type. Rewrite:

> For every natural number n , n plus zero equals n .

Still not Lean. Rewrite:

```
forall n : Nat, n + 0 = n
```

Now a kernel has a chance.

Name the type every time

```
| You said | Lean may hear | Problem |  
| --- | --- | --- |  
| number | Nat, Int, Rat, Real, ... | Different theorems |  
| even | Even n for a specific type | Must import the right Even |  
| prime | Nat prime vs other | Wrong lemmas |
```

If you do not know the type, you are not ready to prove. Learn the type first.

Put the quantifiers in the right order

“There exists a number larger than every number” is false (no largest natural).

“For every number there exists a larger one” is true.

Those are exists n , forall m , ... versus forall n , exists m , The model will swap them. You must not.

Do not hide hypotheses

Bad: $n + n = 2 * n$ with n untyped in your head as “obviously Nat .”

Better: $\text{forall } n : \text{Nat}, n + n = 2 * n$

If you need evenness: $\text{forall } n : \text{Nat}, \text{Even } n \rightarrow \dots$ (only if `Even` elaborates in your imports). If it does not elaborate, the status is rejected, and that is the system working.

Avoid slogans the triviality filter does not catch

The filter only rejects compact `true`, `false`, `trivial`. It will not reject $\text{forall } n : \text{Nat}, \text{True}$ if you write that — wait, `True` as the whole statement is trivial, but $\text{forall } n : \text{Nat}, \text{True}$ is a different string and may typecheck and even prove. That “theorem” is worthless. You are responsible for not asking worthless questions.

ASCII on Windows

Use `forall`, `exists`, `->` or Lean arrows if your console is honest. If a paste inserts a fancy $\forall$ from a website and PowerShell corrupts it, you will get a parse error that looks like Lean is broken. It is not. Retype.

Full declarations versus bare propositions

If your statement starts a line with `theorem`, `lemma`, `example`, or `def`, `render_proof` uses your text as the declaration and does not wrap `theorem aimath_result : (...) := by`. That is for people who already write Lean. Beginners should pass only the proposition. If you pass a half-written theorem `foo : without := by`, the file will be nonsense and Lean will say so.

When Mathlib already has it

If `prove` returns `known`, you wrote a sentence the retriever thinks is in the library. Celebrate the retrieval. Do not edit `sqlite` to force `theorem`. If you wanted the practice of proving it anyway, change the wording just enough that `verbatim compact match` fails — and understand you are then proving a possibly identical fact under a different string. That is training, not discovery.

When you want a relative theorem

Load a system first. Wait for `accepted`. Then `--system ThatName`. If you skip that, you get `ValueError`. If you prove under `Parity` and then tell a friend it is ordinary number theory, you violated Chapter 8.

A checklist before you press Enter

1. Did I name the type?
2. Did I write `forall`/`exists` in the order I mean?
3. Is this `True`/`False`/`trivial`? If yes, stop.
4. Did I type `sorry`? If yes, stop.

5. Am I naming an open problem as if I will solve it? If yes, stop.

6. Are my quotes straight?

7. Is Mathlib ready and is the model up?

Then run `prove` or `search`. Then read the first word of the output before you read the model's vibes.

Appendix E — A worked session, narrated minute by minute

This appendix is a story of one afternoon on a Windows machine. The commands are real. The feelings are typical. The moral is Chapter 1.

14:00 — The folder

You create `D:\work\nat-lab`, activate a venv, `python -m aimath --help` prints the subcommands. You feel nothing yet. Good.

14:05 — lake is missing

`aimath init` prints the elan paragraph. You try the old `irm` one-liner. Connection closed. You feel the internet is broken. It is not. You use `curl.exe` and `elan.lean-lang.org`. You forget to open a new terminal. lake is still missing. You open a new terminal, activate the venv, `lake --version` works. This is the most common human loop. Chapter 10 exists because of it.

14:20 — init

`aimath init` runs a long time. You watch `+ lake update` and `+ lake exe cache get`. You do not `Ctrl+C`. When it finishes, `aimath status` says `mathlib ready: yes`. Disk is smaller. That is Mathlib, not a leak.

14:45 — yaml

You set Ollama, empty `api_key_env`, model name you actually pulled. You forget `ollama serve`. `aimath prove` fails with a connection error. You start Ollama. You retry. Lean may already have typechecked; the failure was the model. You do not `--force init`.

15:00 — first prove

```
aimath prove --statement "forall n : Nat, n + 0 = n" --personality elementary
```

Output first word: `known`. Detail mentions a Mathlib module. You feel cheated. You are not. The system refused to launder a library lemma into a discovery. You run `aimath search --statement "forall n : Nat, n = n"`. Maybe theorem. You open the printed Lean. You see `intro` and `rfl` or just `rfl`. You understand it. That is a good afternoon.

15:20 — explore

```
aimath explore --domain nat --steps 3
```

Three blocks. One known, one rejected (the model said True), one conjecture with a Lean error about a name that does not exist. You do not add an axiom to make the name exist. You write a better domain string: `nat add_zero succ`. Explore again. Slightly better candidates. This is the real job: you improve the question.

15:40 — parity.yaml

You load the example. accepted. You try `aimath prove --system Parity --statement "True"`. rejected (trivial). You feel the system is rude. It is protecting the status word theorem from becoming a joke. You write in your notebook: axioms are assumptions; 0 being even was assumed, not earned, if you used `zero_even` as an axiom even though it is true.

16:00 — report

`aimath report`. You read `reports/report.md` as if you were a teacher who does not like you. You delete a sentence in your own blog draft that said “the AI discovered.” You write “Lean accepted $n = n$ in our workspace.” You close the laptop.

What the afternoon was worth

You now know install minutes, the four statuses, why known is not failure, why True is rejected, and why a long download is the price of a real library. That is more than a demo GIF. The book’s remaining chapters are what you read tonight if you want to change the code tomorrow.